nabia: Architecture, Methodology and Reference

A systematic trend strategy platform: C++ numerical core, Python layer, AI agents and web application

nabia documentation

October 2026

Abstract

This document is the reference for the whole nabia project. It describes the architecture, then every method the project implements, stated mathematically, with proofs where a property is relied on. It covers:

- the C++ numerical core: time series, signal filters, forecasters, five optimizers, risk measures, risk budgeting and the walk-forward portfolio engine;
- the command-line application, the Python bindings and the market data pipeline;
- the AI layer: the three agents with their complete prompts, the MCP servers with every tool's description and input schema, the agent loop, the statistics the agents see, and how news sentiment and relevance are produced and aggregated;
- the web application and the job runner.

Each concept points to the code that implements it, by file and line range, and the key code is shown in excerpts. Excerpts, prompts, tool descriptions and configuration fields are *extracted from the source when the document is built*, so they are the text the programs actually use, and a renamed function stops the build instead of leaving a stale reference.

Contents

I Overview **4**

1 Purpose **4**

2 Architecture **4**
2.1 Layers 4
2.2 Directory map 5
2.3 Build 5

II The numerical core (C++) **6**

3 Time series containers **6**

4 Signal extraction **6**
4.1 Exponentially weighted moving average 7
4.2 Butterworth band-pass 8

5 Forecasting **8**
5.1 Simple exponential smoothing 8
5.2 Kalman filter 9
5.3 Gaussian process regression 10

6 Optimization **11**
6.1 L-BFGS . . . 11
6.2 Augmented Lagrangian . . . 13
6.3 Sequential quadratic programming . . . 14
6.4 Nelder–Mead with box bounds . . . 16
6.5 Differential evolution . . . 17

7 Risk measures **19**

8 Risk budgeting **20**

9 The walk-forward portfolio engine **21**
9.1 Timing and absence of look-ahead . . . 21
9.2 Covariance estimation . . . 22
9.3 Allocation, budget schedule and volatility targeting . . . 23
9.4 Phase 1: parallel solves . . . 24
9.5 Phase 2: drift, trading and accounting . . . 25

10 Performance metrics **26**

III Applications, bindings and data **27**

11 Command-line application **27**

12 Python bindings and pandas API **27**

13 Market data **29**
13.1 Universe and sources . . . 29
13.2 Storage and incremental updates . . . 30

IV The AI layer **31**

14 Agents: roles, data flow and configuration **32**
14.1 Configuration . . . 33

15 MCP servers and tool catalogue **33**
15.1 Design . . . 33
15.2 Catalogue . . . 34
15.3 Server `nabia-news` . . . 35
15.4 Server `nabia-stats` . . . 36
15.5 Server `nabia-portfolio` . . . 37

16 The agent runtime **38**
16.1 Starting an agent . . . 38
16.2 The loop . . . 39
16.3 Providers . . . 41

17 Agent prompts **43**
17.1 Structure and templating . . . 43
17.2 News analyzer . . . 43
17.3 Stats analyzer . . . 44
17.4 Execution agent . . . 44

18 Statistics computed for the agents **46**

19 From decisions to positions **48**
19.1 Budget normalization within bounds 48
19.2 Target weights 49
19.3 The trend rule baseline 49
19.4 Backtest mode 50

20 News: collection, sentiment and relevance **51**
20.1 Collection 51
20.2 How sentiment and relevance are produced 52
20.3 Aggregation 53
20.4 Use and limitations 54

V Operation **54**

21 Web application and job runner **55**

22 Verification **56**

23 Limitations and future work **57**

Part I
Overview

1 Purpose

nabia is a platform for researching and running *systematic trend strategies* across asset classes: equity indices, government bond futures, commodities and crypto. A strategy has four stages, and the project provides a component for each:

1. **Data**: daily prices from public sources into a SQL database, kept up to date incrementally (§13).
2. **Signals and views**:
   - filters and forecasters that extract trends from noisy prices (§4, §5);
   - statistical and news analysis on top of these, done by AI agents (§14–20).
3. **Allocation**: *risk budgeting* decides each asset's share of portfolio risk. A deterministic engine turns budgets into weights, scales them to a volatility target and simulates trading with costs (§8, §9).
4. **Evaluation and operation**: walk-forward backtests without look-ahead, performance metrics, and a web application to run jobs and audit every decision (§10, §21).

The central design decision is to separate *who decides the risk budgets* from *how budgets become positions*:

- Budgets are decided by a fixed rule, by equal risk, or by AI agents.
- Positions come from a tested numerical engine.

Agents never set weights. They choose budgets within bounds, and the engine computes the weights, so an agent's influence is bounded and every decision is reproducible from its stored budgets.

2 Architecture

2.1 Layers

C++ core (`include/nabia`, `src`): the static library `nabia::nabia`, built with CMake. It uses C++20, Eigen for linear algebra and iir1 for filter design. Each module exposes an abstract interface whose parameter struct is owned through `std::unique_ptr`:

- `IFilter<P>` (`include/nabia/signal/IFilter.h:20-43`);
- `IForecaster<P>` (`include/nabia/forecast/IForecaster.h:38-81`);
- `IOptimizer<P,R>` (`include/nabia/optimization/IOptimizer.h:16-39`);
- `IRiskMeasure` (`include/nabia/risk/IRiskMeasure.h:18-43`).

Application `nabia_rb_backtest` (`apps/rb_backtest`): a backtest from a prices CSV and a JSON configuration, written to CSV.

Python bindings (`python/src`): the pybind11 module `nabia._nabia` exposes the engine, and the `nabia` package wraps it in pandas objects.

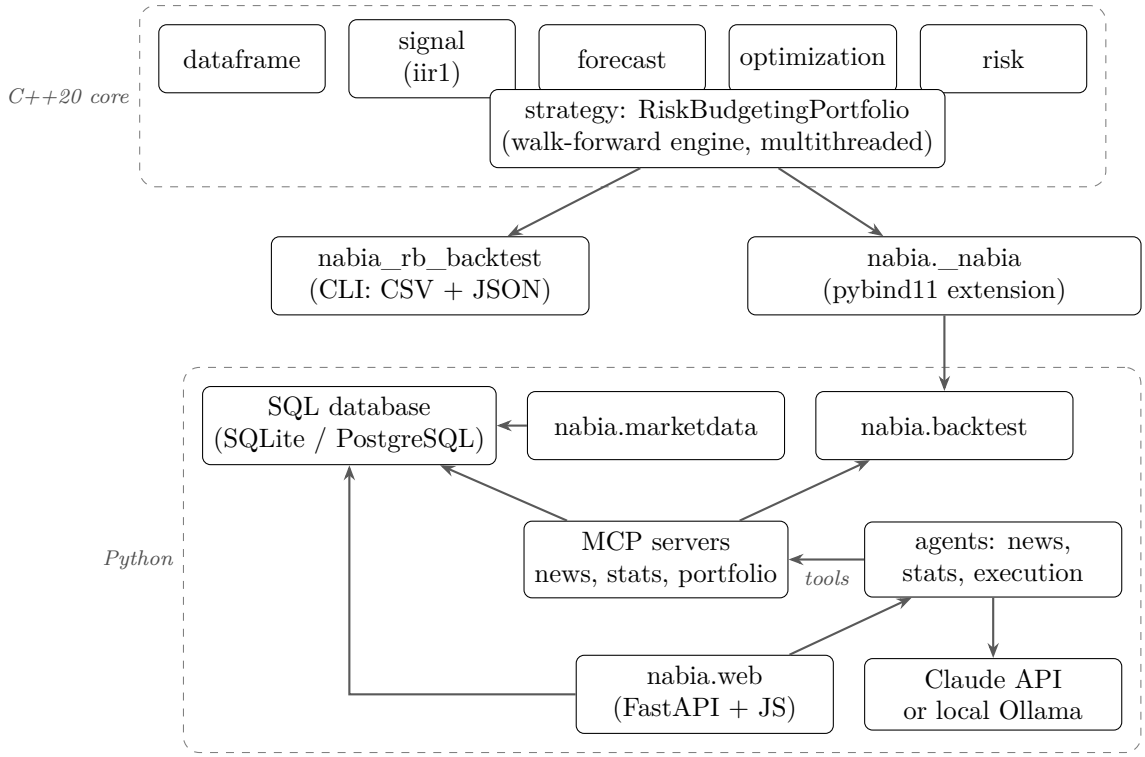


Figure 1: Components and dependencies. Arrows point from a component to what it uses.

Python packages

- `nabia.marketdata`: download and storage;
- `nabia.ai`: agents, MCP servers and pipeline;
- `nabia.web`: web application and job runner.

Storage one SQL database holds prices, news, statistics reports, decisions, agent transcripts, jobs, job logs and backtest runs. Access goes through SQLAlchemy Core with portable types, so moving from SQLite to PostgreSQL changes only the URL.

2.2 Directory map

Path	Contents
`include/nabia/dataframe`	`TimeSeries`, `DataFrame`, the `TimeIndex` concept
`include/nabia/signal`	`IFilter`, `EWMA`, `ButterworthBandPass`, smoothing helpers
`include/nabia/forecast`	`IForecaster`, SES, Kalman filter, Gaussian process, kernels
`include/nabia/optimization`	`IOptimizer`; `convex/`: constraints, L-BFGS, augmented Lagrangian, SQP; `nongradopt/`: Nelder–Mead, differential evolution
`include/nabia/risk`	`IRiskMeasure`, volatility, VaR, ES, `RiskBudgeting<Measure>`
`include/nabia/stats`	descriptive statistics, normal density, CDF and quantile
`include/nabia/strategy`	`RiskBudgetingPortfolio`, performance metrics
`apps/rb_backtest`	command-line backtest (CSV and JSON)
`python/nabia`	`portfolio`, `marketdata`, `ai` (agents, servers, pipeline), `web`
`doc`, `docs`	LaTeX sources (built by CMake) and generated PDFs
`tests`, `python/tests`	Catch2 and pytest suites

2.3 Build

CMake presets (`debug`, `release`, `asan`, `python`) configure the build, with warnings treated as errors. Dependencies are found on the system or fetched: Eigen, iir1, Catch2, nlohmann/json

and pybind11. The documents are built by the `nabia_docs` target. For this document it first runs `doc/project/extract_snippets.py`, which writes the excerpts, the code locations, the tool catalogue of §15 and the configuration table of §14.1.

Part II
The numerical core (C++)

3 Time series containers

All numerical code works on Eigen matrices, but financial data is indexed by time. `TimeSeries<Index>` and `DataFrame<Index>` pair an Eigen vector or matrix (rows are times) with a *strictly increasing* index:

- The index type satisfies the `TimeIndex` concept (copyable and totally ordered): calendar dates, timestamps or integers.
- Construction validates the shapes, the ordering and unique column names.
- Time lookups use binary search.

Alignment. Markets have different holidays. A cross-asset backtest keeps only the dates where every asset has a finite price.

Returns. Simple returns $r_t = P_t/P_{t-1} - 1$ (`pct_change`) or log returns $\log(P_t/P_{t-1})$ are computed between consecutive *kept* rows. A return can therefore span a removed holiday. That is the right convention for a portfolio that can only trade when all its markets are open.

Code 1: Alignment: keep rows where every column is finite
`include/nabia/dataframe/DataFrame.h:108-122`

```
[[nodiscard]] DataFrame drop_missing() const {
    std::vector<Index> index;
    std::vector<Eigen::Index> keep;
    for (Eigen::Index i = 0; i < rows(); ++i) {
        if (values_.row(i).allFinite()) {
            keep.push_back(i);
            index.push_back(index_[static_cast<std::size_t>(i)]);
        }
    }
    Eigen::MatrixXd out(static_cast<Eigen::Index>(keep.size()), cols());
    for (std::size_t k = 0; k < keep.size(); ++k) {
        out.row(static_cast<Eigen::Index>(k)) = values_.row(keep[k]);
    }
    return DataFrame(std::move(index), columns_, std::move(out));
}
```

Code

`TimeIndex` concept — `include/nabia/dataframe/TimeIndex.h:17`
`TimeSeries` — `include/nabia/dataframe/TimeSeries.h:47-106`
`DataFrame` — `include/nabia/dataframe/DataFrame.h:23-146`
simple returns — `include/nabia/dataframe/DataFrame.h:126-132`
log returns — `include/nabia/dataframe/DataFrame.h:134-140`

4 Signal extraction

Filters implement `IFilter<P>`. They are *causal*: output t uses inputs up to t only. Two modes are supported:

- streaming: `process(x)`, one sample at a time, keeping state;
- batch: `apply(signal)`, from a fresh state, without disturbing the streaming state.

Filter design is delegated to iir1, behind a private implementation struct (pimpl), so iir1 never appears in public headers.

4.1 Exponentially weighted moving average

The EWMA is the one-pole low-pass

$$y_t = \alpha x_t + (1-\alpha) y_{t-1}, \qquad H(z) = \frac{\alpha}{1-(1-\alpha)z^{-1}}, \qquad \alpha \in (0, 1].$$

Its DC gain is $H(1) = 1$, so a constant input passes unchanged.

Proposition 4.1 (Seeding)**.** *Let $\tilde{y}$ be the EWMA of $x_t - x_0$ started from a zero state. Then $y_t = x_0 + \tilde{y}_t$ equals the recursion started at $y_{-1} = x_0$:*

$$y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} x_0.$$

Proof. Unrolling the recursion from y_{-1} gives $y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} y_{-1}$. With a zero state, $\tilde{y}_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k (x_{t-k} - x_0)$. Using $\alpha \sum_{k=0}^{t} (1-\alpha)^k = 1 - (1-\alpha)^{t+1}$, adding x_0 gives the formula with $y_{-1} = x_0$. □

Starting from zero instead would bias the early outputs towards zero, so the filter seeds with the first sample by default.

Parameterizations. Two helpers convert the usual parameterizations to α:

- `alpha_from_span`$(N) = 2/(N+1)$. The mean age of the EWMA weights is $\sum_k k\,\alpha(1-\alpha)^k = (1-\alpha)/\alpha$, and that of an N-point moving average is $(N-1)/2$. Equating the two gives $\alpha = 2/(N+1)$.
- `alpha_from_halflife`$(h) = 1 - 2^{-1/h}$. This solves $(1-\alpha)^h = \frac{1}{2}$: the weight h steps back is half the newest weight.

Code 2: EWMA on iir1's one-pole section, with first-sample seeding
`src/signal/EWMA.cpp:16-36`

```
struct EWMA::Impl {
    Iir::Custom::OnePole section;
    Iir::DirectFormI state;
    bool seed = true;
    bool started = false;
    double offset = 0.0;

    double process(double x) {
        if (!started) {
            offset = seed ? x : 0.0;
            started = true;
        }
        return section.filter(x - offset, state) + offset;
    }

    void reset() {
        state.reset();
        started = false;
        offset = 0.0;
    }
};
```

Code

class and parameters — `include/nabia/signal/EWMA.h:23-43`
span and half-life conversions — `src/signal/EWMA.cpp:72-79`
simple moving average helper (rolling sum, $O(n)$) — `src/signal/smoothing.cpp:7-28`

4.2 Butterworth band-pass

A causal Butterworth band-pass isolates trend cycles between two frequencies $0 < f_\ell < f_h < f_s/2$. It is built in three steps.

1. **Analog prototype.** The order-N low-pass prototype is maximally flat:

$$|H(j\Omega)|^2 = \frac{1}{1+\Omega^{2N}},$$

   with poles $s_k = e^{j\pi(2k+N-1)/(2N)}$, $k = 1, \dots, N$, on the left half of the unit circle.

2. **Low-pass to band-pass.** The substitution

$$s \mapsto \frac{s^2+\Omega_0^2}{sB}$$

   maps it to a band-pass with $2N$ poles, centred on Ω_0 with bandwidth B.

3. **Bilinear transform.** The map $s = 2f_s\frac{1-z^{-1}}{1+z^{-1}}$, with the band edges prewarped by $\Omega = 2f_s\tan(\pi f/f_s)$, gives the digital filter. It is run as a cascade of second-order sections, which is numerically stable at high orders.

iir1 takes the band as a centre and a width, so the edges map to centre $(f_\ell + f_h)/2$ and width $f_h - f_\ell$. The filter then has $|H| = 1/\sqrt{2}$ ($-3\,\mathrm{dB}$) at both edges and unit gain at the centre. The tests check exactly these properties, and strong attenuation outside the band. Being causal, the filter delays the signal: the group delay grows with the order and with narrower bands.

Code

parameters (order, sample rate, edges) — `include/nabia/signal/ButterworthBandPass.h:11-16`
design: edges to centre and width — `src/signal/ButterworthBandPass.cpp:19-22`

5 Forecasting

Forecasters implement `IForecaster<P>`:

- `fit(y)` estimates the model on a series;
- `forecast(H)` returns the mean $\hat{y}_{T+h}$ and the predictive variance v_h for $h = 1..H$;
- Gaussian prediction intervals follow as $\hat{y} \pm z_{(1+c)/2}\sqrt{v_h}$.

The companion documents in `doc/forecast` derive each model in full. This section states what the code computes.

5.1 Simple exponential smoothing

The model is the ETS(A,N,N) state space

$$y_t = \ell_{t-1} + \varepsilon_t, \qquad \ell_t = \ell_{t-1} + \alpha\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2).$$

The fit initializes $\ell = y_1$ and runs the error-correction form $\ell \leftarrow \ell + \alpha(y_t - \ell)$, accumulating $\mathrm{SSE} = \sum_t (y_t - \ell_{t-1})^2$. It then estimates $\hat{\sigma}^2 = \mathrm{SSE}/T$.

Proposition 5.1. *The h-step forecast is $\hat{y}_{T+h} = \ell_T$, with variance $\sigma^2(1 + (h-1)\alpha^2)$.*

Proof. Iterating the state equation gives $\ell_{T+h-1} = \ell_T + \alpha \sum_{j=1}^{h-1} \varepsilon_{T+j}$, so $y_{T+h} = \ell_T + \alpha \sum_{j=1}^{h-1} \varepsilon_{T+j} + \varepsilon_{T+h}$. The errors are independent, so the variance is $(h-1)\alpha^2\sigma^2 + \sigma^2$. □

Code 3: SES fit and forecast variance
`src/forecast/exponential_smoothing.cpp:27-45`

```
    // Initialise the level with the first observation.
    level_ = y(0);
    for (Eigen::Index t = 0; t < y.size(); ++t) {
        in_sample_(t) = level_;
        const double err = y(t) - level_;
        sse_ += err * err;
        level_ += alpha * err; // equivalent to alpha*y + (1-alpha)*level
    }
    fitted_ = true;
}

Forecast SimpleExponentialSmoothing::forecast(Eigen::Index horizon) const {
    require_fitted("SimpleExponentialSmoothing::forecast");
    require_horizon(horizon, "SimpleExponentialSmoothing::forecast");
    const double alpha = parameters_->alpha;
    const double sigma2 = sse_ / static_cast<double>(in_sample_.size());
    Forecast out{Eigen::VectorXd::Constant(horizon, level_), Eigen::VectorXd(horizon)};
    for (Eigen::Index h = 0; h < horizon; ++h) {
        out.variance(h) = sigma2 * (1.0 + static_cast<double>(h) * alpha * alpha);
```

5.2 Kalman filter

The model is the linear Gaussian state space

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q); \qquad y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R),$$

with prior $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$, where the prior describes the first state. Each step first predicts:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \qquad P_{t|t-1} = FP_{t-1|t-1}F^\top + Q.$$

Then it computes the innovation and its variance:

$$e_t = y_t - H\hat{x}_{t|t-1}, \qquad S_t = HP_{t|t-1}H^\top + R.$$

Finally it corrects, with gain $K_t = P_{t|t-1}H^\top / S_t$:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \qquad P_{t|t} = (I - K_t H)P_{t|t-1}(I - K_t H)^\top + R\,K_t K_t^\top.$$

- **Joseph form.** The covariance update above is the posterior covariance for *any* gain. It is symmetric and positive semi-definite by construction, so it stays valid under rounding, unlike the short form $(I - KH)P$.
- **Missing observations.** Non-finite y_t skip the correction.
- **Log-likelihood.** $\ell = -\frac{1}{2}\sum_t \left(\log 2\pi + \log S_t + e_t^2/S_t\right)$, summed over the observed points.
- **Forecasts.** They iterate the prediction step: mean Hx, variance $HPH^\top + R$.
- **Smoothing.** The Rauch–Tung–Striebel smoother uses the gain $C_t = P_{t|t}F^\top P_{t+1|t}^{-1}$ (computed by an $LDL^\top$ solve, not an inverse) and $\hat{x}_t^s = \hat{x}_{t|t} + C_t(\hat{x}_{t+1}^s - \hat{x}_{t+1|t})$.

Two ready-made models are provided:

- *local level*: $F = H = 1$, so the level is a random walk observed with noise;
- *local linear trend*: $F = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$, $H = (1, 0)$, $Q = \mathrm{diag}(q_\ell, q_s)$, so level and slope both drift.

Both use a diffuse prior ($P_0 = 10^7 I$). The local linear trend's slope state is a direct estimate of the trend.

Code 4: Kalman predict and Joseph-form correction
`src/forecast/KalmanFilter.cpp:91-117`

```
if (t > 0) {  // predict; the prior already describes x[0]
    x = F * x;
    P = F * P * F.transpose() + p.process_noise;
}
const auto ts = static_cast<std::size_t>(t);
predicted_states_.col(t) = x;
predicted_covariances_[ts] = P;

const double y_hat = H.dot(x);
const VectorXd PHt = P * H.transpose();
const double S = H.dot(PHt) + p.observation_noise;
one_step_(t) = y_hat;
innovation_variances_(t) = S;

if (std::isfinite(y(t))) {  // correct
    const double e = y(t) - y_hat;
    const VectorXd K = PHt / S;
    x += K * e;
    const MatrixXd A = I - K * H;
    P = A * P * A.transpose() + p.observation_noise * K * K.transpose();
    innovations_(t) = e;
    log_likelihood_ -= 0.5 * (log_2pi + std::log(S) + e * e / S);
} else {
    innovations_(t) = std::numeric_limits<double>::quiet_NaN();
}
filtered_states_.col(t) = x;
filtered_covariances_[ts] = P;
```

Code

parameters F, H, Q, R, x_0, P_0 — `include/nabia/forecast/KalmanFilter.h:16-34`
local level and local linear trend — `src/forecast/KalmanFilter.cpp:25-48`
forecast — `src/forecast/KalmanFilter.cpp:122-136`
RTS smoother — `src/forecast/KalmanFilter.cpp:168-182`

5.3 Gaussian process regression

The series is regressed on time $t = 0, \ldots, T-1$ in three steps.

1. **Standardization.** The observations are standardized, $z = (y - \bar{y})/s$, where s is the sample standard deviation when normalizing.

2. **Model.** $z \sim \mathcal{N}(0, K)$ with $K = K_f + \sigma_n^2 I$ and $(K_f)_{ij} = k(t_i, t_j)$. The available kernels, with $r = t - t'$:

$$
\begin{aligned}
k_{\mathrm{SE}} &= \sigma^2 \exp(-r^2/2\ell^2), \\
k_{\mathrm{M52}} &= \sigma^2 (1 + u + u^2/3) e^{-u}, \quad u = \sqrt{5}|r|/\ell, \\
k_{\mathrm{per}} &= \sigma^2 \exp(-2\sin^2(\pi r/p)/\ell^2).
\end{aligned}
$$

   Sums and products of kernels are kernels too.

3. **Hyperparameters.** With $\alpha = K^{-1} z$ (from a Cholesky factorization $K = LL^\top$), the log marginal likelihood is

$$
\log p(z) = -\tfrac{1}{2} z^\top \alpha - \textstyle\sum_i \log L_{ii} - \tfrac{T}{2} \log 2\pi.
$$

The hyperparameters θ (logs of variances, length scales, periods and, optionally, the noise) maximize it, using L-BFGS with the analytic gradient

$$\frac{\partial(-\log p)}{\partial \theta_j} = -\tfrac{1}{2}\operatorname{tr}\Big(\big(\alpha\alpha^\top - K^{-1}\big)\frac{\partial K}{\partial \theta_j}\Big).$$

Working in logs keeps the parameters positive. Values with $|\theta_j| > 20$, or a K that is not positive definite, are returned as $+\infty$, which the line search rejects. The noise has a floor (`min_noise_variance`).

Prediction. At new times t_*, with $v = L^{-1}k_*$:

$$\text{mean} = \bar{y} + s\,k_*^\top \alpha, \qquad \text{variance} = s^2\big(k(t_*,t_*) - v^\top v + \sigma_n^2\big).$$

The reported likelihood is converted back to the units of y by subtracting $T \log s$.

Code 5: Gaussian process: negative log-likelihood gradient
`src/forecast/GaussianProcess.cpp:100-111`

```
// d(-log p)/d theta_j = -1/2 tr((alpha alpha' - K^-1) dK/d theta_j)
const MatrixXd W = f->alpha * f->alpha.transpose() - f->llt.solve(MatrixXd::Identity(T, T));
grad.setZero();
for (Index j = 0; j < T; ++j) {
    for (Index i = 0; i < T; ++i) {
        grad.head(nk) -= 0.5 * W(i, j) * work->gradient(times_(i), times_(j));
    }
}
if (p.optimize_noise) {
    grad(nk) = -0.5 * noise * W.trace();  // dK/dlog(noise) = noise I
}
return -f->log_ml;
```

Code

Cholesky and log marginal likelihood — `src/forecast/GaussianProcess.cpp:31-42`
fit with hyperparameter optimization — `src/forecast/GaussianProcess.cpp:56-133`
predictive mean and variance — `src/forecast/GaussianProcess.cpp:135-147`
kernels and their log-parameter gradients — `src/forecast/Kernels.cpp:41-226`

6 Optimization

Optimizers implement `IOptimizer<P,R>`: `optimize(initial_guess)` returns the solution, and `result()` returns a solver-specific result object (status, iterations, residuals). Linear constraints are expressed in matrix form: $A_{\text{eq}}x = b_{\text{eq}}$ and $A_{\text{in}}x \le b_{\text{in}}$. Box bounds $l \le x \le u$ convert to inequality rows.

Code

optimizer interface — `include/nabia/optimization/IOptimizer.h:16-39`
linear constraints and their conversion — `include/nabia/optimization/convex/Constraints.h:22-47`

6.1 L-BFGS

L-BFGS minimizes a smooth f with search direction $d = -H_k g_k$. Here H_k is the BFGS inverse-Hessian approximation built from the last m pairs $s_j = x_{j+1} - x_j$, $y_j = g_{j+1} - g_j$, with $\rho_j = 1/s_j^\top y_j$. It is applied in $O(mn)$ by the two-loop recursion:

1. $q \leftarrow g$. For j from newest to oldest: $a_j = \rho_j s_j^\top q$, then $q \leftarrow q - a_j y_j$.
2. $r \leftarrow \gamma q$, with $\gamma = s^\top y / y^\top y$ from the newest pair. On the first iteration $\gamma = 1/\max(1, \|g\|)$.

3. For j from oldest to newest: $\beta = \rho_j y_j^\top r$, then $r \leftarrow r + (a_j - \beta)s_j$. Finally $d = -r$.

Safeguards keep the method well defined:

- A pair is stored only if $s^\top y > 10^{-12}\|s\|\|y\|$ (the curvature condition), which keeps H positive definite.
- If d is not a descent direction, the memory is cleared and the method restarts from steepest descent.
- Iteration stops when $\|g\|_\infty \le \varepsilon$.

Line search. Backtracking from $t = 1$ halves t until the step is accepted. A non-finite f counts as rejection, which is how barrier and domain constraints are enforced. A step is accepted under either condition:

Armijo $f(x + td) \le f(x) + c_1 t\, g^\top d$, with $c_1 = 10^{-4}$.

Approximate Wolfe (Hager–Zhang), when f is flat to within rounding: $f(x + td) \le f(x) + \epsilon_f |f(x)|$ and $\sigma\, g^\top d \le g(x + td)^\top d \le (2c_1 - 1)\, g^\top d$, with $\epsilon_f = 10^{-10}$ and $\sigma = 0.9$.

Near a minimum the decrease Armijo requires is smaller than the rounding error in f, while directional derivatives remain accurate. The second test lets the method keep converging there. On 500-asset risk budgeting it took the attainable budget accuracy from about $3 \cdot 10^{-8}$ to 10^{-8}.

Code 6: Two-loop recursion
`src/optimization/convex/LBFGS.cpp:63-77`

```cpp
// Two-loop recursion: d = -H g.
VectorXd q = g;
std::vector<double> alpha(s_hist.size());
for (std::size_t j = s_hist.size(); j-- > 0;) {
    alpha[j] = rho_hist[j] * s_hist[j].dot(q);
    q -= alpha[j] * y_hist[j];
}
const double gamma = s_hist.empty() ? 1.0 / std::max(1.0, g.norm())
                                    : s_hist.back().dot(y_hist.back()) / y_hist.back().squaredNorm();
VectorXd d = gamma * q;
for (std::size_t j = 0; j < s_hist.size(); ++j) {
    const double beta = rho_hist[j] * y_hist[j].dot(d);
    d += (alpha[j] - beta) * s_hist[j];
}
d = -d;
```

Code 7: Line search: Armijo or approximate Wolfe
`src/optimization/convex/LBFGS.cpp:88-116`

```cpp
double t = 1.0;
VectorXd x_new;
VectorXd g_new(x.size());
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < p.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = value_and_gradient_(x_new, g_new);
    ++result_.evaluations;
    if (!std::isfinite(f_new)) {
        t *= p.backtrack;
        continue;
    }
    // Armijo sufficient decrease, or Hager-Zhang's approximate Wolfe
    // conditions when f is flat to within rounding: near the minimum
    // the decrease in f drops below machine precision, but the
    // directional derivative is still accurate.
    const double new_slope = g_new.dot(d);
    const bool armijo = f_new <= fx + p.armijo_c1 * t * slope;
```

```
    const bool approx_wolfe = f_new <= fx + p.flat_tolerance * std::abs(fx) &&
                              new_slope <= (2.0 * p.armijo_c1 - 1.0) * slope &&
                              new_slope >= p.wolfe_c2 * slope;
    if (armijo || approx_wolfe) {
        accepted = true;
        break;
    }
    t *= p.backtrack;
}
if (!accepted) {
```

Code

parameters and defaults — `include/nabia/optimization/convex/LBFGS.h:12-25`
full algorithm — `src/optimization/convex/LBFGS.cpp:30-148`

6.2 Augmented Lagrangian

For $\min f(x)$ subject to $h(x) = A_{\text{eq}}x - b_{\text{eq}} = 0$ and $g(x) = A_{\text{in}}x - b_{\text{in}} \le 0$, the Powell–Hestenes–Rockafellar augmented Lagrangian is

$$L_\rho(x;\lambda,\mu) = f(x) + \lambda^\top h + \frac{\rho}{2}\|h\|^2 + \frac{1}{2\rho}\sum_i \Big(\max(0,\mu_i + \rho g_i)^2 - \mu_i^2\Big),$$

with gradient $\nabla f + A_{\text{eq}}^\top(\lambda + \rho h) + A_{\text{in}}^\top \max(0, \mu + \rho g)$. Each outer iteration does four things:

1. **Minimize** L_ρ with L-BFGS, warm-started at the previous iterate. The inner tolerance starts at $\max(\varepsilon, 0.1)$ and shrinks tenfold per outer iteration down to ε.
2. **Measure** the violation $v = \max\left(\|h\|_\infty, \|\max(g, -\mu/\rho)\|_\infty\right)$. The second term is zero if and only if $g \le 0$ and $\mu_i g_i = 0$, so it also measures complementarity.
3. **Update the multipliers**: $\lambda \leftarrow \lambda + \rho h$ and $\mu \leftarrow \max(0, \mu + \rho g)$. After this update, $\nabla_x L_\rho = \nabla f + A_{\text{eq}}^\top\lambda + A_{\text{in}}^\top\mu$, so the inner gradient norm is the KKT stationarity.
4. **Test and adapt.** Stop when $v \le \varepsilon$ and stationarity $\le \varepsilon$. Otherwise, if $v > 0.25\, v_{\text{prev}}$, set $\rho \leftarrow \min(10\rho, 10^{10})$ (starting from $\rho_0 = 10$).

Code 8: Augmented Lagrangian value and gradient
`src/optimization/convex/AugmentedLagrangian.cpp:58-73`

```
// L_rho(x) = f + lambda'h + rho/2 |h|^2 + 1/(2 rho) sum(max(0, mu + rho g)^2 - mu^2)
// grad     = grad f + A_eq'(lambda + rho h) + A_ineq' max(0, mu + rho g)
const LBFGS::ValueAndGradient augmented = [&](const VectorXd& z, VectorXd& grad) {
    double value = objective_(z);
    grad = gradient_(z);
    if (has_eq) {
        const VectorXd h = lf.A_eq * z - lf.b_eq;
        value += lambda.dot(h) + 0.5 * rho * h.squaredNorm();
        grad.noalias() += lf.A_eq.transpose() * (lambda + rho * h);
    }
    if (has_ineq) {
        const VectorXd shifted = (mu + rho * (lf.A_ineq * z - lf.b_ineq)).cwiseMax(0.0);
        value += (shifted.squaredNorm() - mu.squaredNorm()) / (2.0 * rho);
        grad.noalias() += lf.A_ineq.transpose() * shifted;
    }
    return value;
```

Code 9: Multiplier and penalty updates
`src/optimization/convex/AugmentedLagrangian.cpp:93-119`

```
// Violation measured before the multiplier update; for inequalities
// max(g, -mu/rho) is zero iff g <= 0 and mu * g = 0 (complementarity).
double violation = 0.0;
```

```
if (has_eq) {
    const VectorXd h = lf.A_eq * x - lf.b_eq;
    violation = h.lpNorm<Eigen::Infinity>();
    lambda += rho * h;
}
if (has_ineq) {
    const VectorXd g = lf.A_ineq * x - lf.b_ineq;
    violation = std::max(violation, g.cwiseMax(-mu / rho).lpNorm<Eigen::Infinity>());
    mu = (mu + rho * g).cwiseMax(0.0);
}

// After the update, grad L_rho(x) = grad f + A_eq' lambda + A_ineq' mu.
result_.stationarity = inner.gradient_norm;
result_.constraint_violation = violation;
if (violation <= p.tolerance && inner.gradient_norm <= p.tolerance) {
    result_.converged = true;
    break;
}

if (violation > options_.required_decrease * previous_violation) {
    rho = std::min(rho * options_.penalty_growth, options_.max_penalty);
}
previous_violation = violation;
inner_tol = std::max(p.tolerance, 0.1 * inner_tol);
```

Code

options and defaults — `include/nabia/optimization/convex/AugmentedLagrangian.h:15-23`
full algorithm — `src/optimization/convex/AugmentedLagrangian.cpp:33-128`

6.3 Sequential quadratic programming

At x_k, SQP solves the quadratic subproblem

$$\min_d \; g_k^\top d + \tfrac{1}{2} d^\top B_k d \quad \text{s.t.} \quad A_{\text{eq}} d = b_{\text{eq}} - A_{\text{eq}} x_k, \;\; A_{\text{in}} d \le b_{\text{in}} - A_{\text{in}} x_k.$$

It uses the augmented Lagrangian of §6.2 at tolerance 10^{-10}, or a Cholesky solve $d = -B^{-1} g$ when there are no constraints.

Convergence is declared when three KKT residuals at x_k, computed with the QP multipliers, are all below ε:

- stationarity $\|g + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu\|_\infty$;
- constraint violation;
- complementarity $\max_i |\mu_i g_i|$.

Merit function $\phi(x) = f(x) + \nu V_1(x)$, with $V_1 = \|h\|_1 + \sum_i \max(g_i, 0)$. The penalty is kept at $\nu \ge \|(\lambda, \mu)\|_\infty + 10^{-3}$. Because the constraints are linear and d satisfies the linearized constraints, $V_1(x + td) \le (1 - t) V_1(x)$. The directional derivative is therefore bounded by $g^\top d - \nu V_1(x) < 0$, so d descends ϕ. Backtracking applies Armijo to ϕ with this slope.

Restart If the line search fails, B resets to the identity. A failure from $B = I$ ends the iteration.

Proposition 6.1 (Damped BFGS, Powell)**.** *Let $r = \theta y + (1 - \theta) Bs$, with $\theta = 1$ if $s^\top y \ge 0.2\, s^\top Bs$ and $\theta = 0.8\, s^\top Bs / (s^\top Bs - s^\top y)$ otherwise. The update*

$$B^+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{rr^\top}{s^\top r}$$

keeps B positive definite.

Proof. If $\theta < 1$, then $s^\top r = s^\top Bs - \theta(s^\top Bs - s^\top y) = 0.2\, s^\top Bs > 0$. If $\theta = 1$, then $s^\top r = s^\top y \geq 0.2\, s^\top Bs > 0$. The BFGS update with a pair satisfying $s^\top r > 0$ preserves positive definiteness. □

After the first step with $s^\top y > 0$, B is scaled to $(y^\top y / s^\top y) I$ before the update, which matches the curvature along s.

Code 10: Damped BFGS update
`src/optimization/convex/SQP.cpp:41-53`

```
void damped_bfgs_update(MatrixXd& B, const VectorXd& s, const VectorXd& y) {
    const VectorXd Bs = B * s;
    const double sBs = s.dot(Bs);
    if (!(sBs > 0.0)) {
        return;
    }
    const double sy = s.dot(y);
    const double theta = sy >= 0.2 * sBs ? 1.0 : 0.8 * sBs / (sBs - sy);
    const VectorXd r = theta * y + (1.0 - theta) * Bs;
    B.noalias() -= Bs * Bs.transpose() / sBs;
    B.noalias() += r * r.transpose() / s.dot(r);
    B = 0.5 * (B + B.transpose());  // remove round-off asymmetry
}
```

Code 11: ℓ_1 merit penalty and line search
`src/optimization/convex/SQP.cpp:156-187`

```
// l1 merit function phi = f + nu * V1; nu must exceed the multipliers
// for d to be a descent direction of phi.
double max_multiplier = 0.0;
if (lambda.size() > 0) {
    max_multiplier = lambda.lpNorm<Eigen::Infinity>();
}
if (mu.size() > 0) {
    max_multiplier = std::max(max_multiplier, mu.lpNorm<Eigen::Infinity>());
}
nu = std::max(nu, max_multiplier + options_.merit_penalty_margin);
const double v1 = l1_violation(x);
const double merit = fx + nu * v1;
// Directional derivative of phi along d (the constraints are linear and
// d satisfies them, so V1(x + t d) <= (1 - t) V1(x)).
double slope = gx.dot(d) - nu * v1;
if (!(slope < 0.0)) {
    slope = -1e-16 * std::max(1.0, std::abs(merit));  // d ~ 0: accept the first decrease
}

double t = 1.0;
VectorXd x_new;
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < options_.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = objective_(x_new);
    ++result_.evaluations;
    if (std::isfinite(f_new) && f_new + nu * l1_violation(x_new) <= merit + options_.merit_c1 * t *
    ↪ slope) {
        accepted = true;
        break;
    }
    t *= options_.backtrack;
```

Code

options and defaults — `include/nabia/optimization/convex/SQP.h:15-23`
QP subproblem through the augmented Lagrangian — `src/optimization/convex/SQP.cpp:26-38`
full algorithm — `src/optimization/convex/SQP.cpp:66-218`

6.4 Nelder–Mead with box bounds

Nelder–Mead keeps a simplex of $n+1$ vertices sorted by value, $f(x_0) \le \cdots \le f(x_n)$, with c the centroid of the best n. Each iteration:

1. **Reflect**: $x_r = c + \alpha(c - x_n)$.
2. If $f_r < f_0$, **expand** to $x_e = c + \gamma(x_r - c)$ and keep the better of x_e and x_r.
3. Else, if $f_r < f_{n-1}$, accept x_r.
4. Else **contract**:
   - outside, $x_c = c + \rho(x_r - c)$, if $f_r < f_n$ (accepted if $f_c < f_r$);
   - inside, $x_c = c + \rho(x_n - c)$, otherwise (accepted if $f_c < f_n$).
5. Else **shrink** every vertex towards the best: $x_j \leftarrow x_0 + \sigma(x_j - x_0)$.

The *adaptive* coefficients of Gao and Han, $\alpha = 1$, $\gamma = 1 + 2/n$, $\rho = \frac{3}{4} - \frac{1}{2n}$, $\sigma = 1 - \frac{1}{n}$, avoid the stagnation of the standard values $(1, 2, \frac{1}{2}, \frac{1}{2})$ in higher dimensions.

Bounds are enforced by projecting every trial point onto the box. The box is convex, so a shrink stays inside it.

Termination. The iteration stops when both conditions hold:

$$f_n - f_0 \le \tau_f(1 + |f_0|) \quad \text{and} \quad \max_j \|x_j - x_0\|_\infty \le \tau_x(1 + \|x_0\|_\infty).$$

A NaN objective counts as $+\infty$.

Code 12: One Nelder–Mead iteration
`src/optimization/nongradopt/NelderMead.cpp:119-154`

```
const VectorXd centroid = simplex.leftCols(n).rowwise().mean();
const VectorXd worst = simplex.col(n);
const VectorXd xr = box.project(centroid + alpha * (centroid - worst));
const double fr = f(xr);

if (fr < values(0)) {
    const VectorXd xe = box.project(centroid + gamma * (xr - centroid));
    const double fe = f(xe);
    if (fe < fr) {
        simplex.col(n) = xe;
        values(n) = fe;
    } else {
        simplex.col(n) = xr;
        values(n) = fr;
    }
    continue;
}
if (fr < values(n - 1)) {
    simplex.col(n) = xr;
    values(n) = fr;
    continue;
}
// Contraction: outside if the reflection beat the worst vertex, inside otherwise.
const bool outside = fr < values(n);
const VectorXd xc = outside ? VectorXd(box.project(centroid + rho * (xr - centroid)))
                            : VectorXd(box.project(centroid + rho * (worst - centroid)));
const double fc = f(xc);
if (fc < (outside ? fr : values(n))) {
    simplex.col(n) = xc;
    values(n) = fc;
    continue;
}
// Shrink towards the best vertex (stays inside the box: the box is convex).
for (Index j = 1; j <= n; ++j) {
    simplex.col(j) = simplex.col(0) + sigma * (simplex.col(j) - simplex.col(0));
    values(j) = f(simplex.col(j));
```

Code

adaptive coefficients — `src/optimization/nongradopt/NelderMead.cpp:43-46`
box projection — `src/optimization/nongradopt/box.hpp:17-23`
full algorithm, initial simplex — `src/optimization/nongradopt/NelderMead.cpp:24-162`

6.5 Differential evolution

Population. N_p vectors (default $15n$, at least 4, or 6 for rand/2), drawn uniformly in the box. An initial guess, if given, replaces member 0.

Mutation. Each generation draws one $F \sim U[F_{\min}, F_{\max}]$ (dithering), then builds a mutant per member i from distinct random members $r_1, \ldots, r_5 \neq i$:

$$\begin{aligned}
\text{rand/1:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}), \\
\text{best/1:} \quad & v = x_{\text{best}} + F(x_{r_1} - x_{r_2}), \\
\text{current-to-best/1:} \quad & v = x_i + F(x_{\text{best}} - x_i) + F(x_{r_1} - x_{r_2}), \\
\text{rand/2:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}) + F(x_{r_4} - x_{r_5}).
\end{aligned}$$

Crossover. Binomial: $u_{ij} = v_{ij}$ if $U < \text{CR}$ or $j = j_{\text{rand}}$, else x_{ij}, so at least one coordinate comes from the mutant. A coordinate outside the box bounces back to the midpoint between the parent and the violated bound.

Selection. Greedy: u replaces x_i if $f(u) \leq f(x_i)$. Ties go to the trial, which keeps the population moving on plateaus.

Termination. The run stops when the population's objective values have $\text{sd}(f) \leq \tau_a + \tau|\overline{f}|$.

Parallelism and reproducibility. All trial vectors of a generation are built sequentially from one seeded RNG *before* evaluation. Only the evaluations run in parallel, so the result is the same for every thread count. A `ParallelEvaluator<T,R>` does the evaluations:

- each input becomes a `std::packaged_task` in a queue;
- n worker threads consume the queue;
- the caller collects the results, in input order, through the tasks' futures.

Code 13: Mutation and crossover of one generation
`src/optimization/nongradopt/DifferentialEvolution.cpp:132-172`

```
const double F = p.mutation_min + (p.mutation_max - p.mutation_min) * unit(rng);

// Build all trial vectors first (sequential RNG => thread-count independent).
for (Index j = 0; j < np; ++j) {
    VectorXd mutant(n);
    switch (p.strategy) {
    case DEStrategy::Rand1: {
        const auto r = pick_distinct<3>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2]));
        break;
    }
    case DEStrategy::Best1: {
        const auto r = pick_distinct<2>(rng, np, j);
        mutant = population.col(best) + F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::CurrentToBest1: {
        const auto r = pick_distinct<2>(rng, np, j);
```

```
        mutant = population.col(j) + F * (population.col(best) - population.col(j)) +
                 F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::Rand2: {
        const auto r = pick_distinct<5>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2])) +
                 F * (population.col(r[3]) - population.col(r[4]));
        break;
    }
    }

    // Binomial crossover; coordinate j_rand always comes from the mutant.
    const Index j_rand = any_coordinate(rng);
    for (Index i = 0; i < n; ++i) {
        double v = (i == j_rand || unit(rng) < p.crossover) ? mutant(i) : population(i, j);
        // Bounce back halfway between the parent and the violated bound.
        if (v < box.lower(i)) {
            v = 0.5 * (box.lower(i) + population(i, j));
        } else if (v > box.upper(i)) {
            v = 0.5 * (box.upper(i) + population(i, j));
        }
        trials(i, j) = v;
```

Code 14: ParallelEvaluator: job queue, consumers, futures
`src/optimization/nongradopt/parallel_evaluator.hpp:33-76`

```
[[nodiscard]] std::vector<TResult> evaluate(const std::vector<T>& inputs) const {
    std::queue<std::packaged_task<TResult()>> jobs;
    std::vector<std::future<TResult>> futures;
    futures.reserve(inputs.size());
    for (const T& input : inputs) {
        std::packaged_task<TResult()> job([this, &input] { return objective_(input); });
        futures.push_back(job.get_future());
        jobs.push(std::move(job));
    }

    std::mutex mutex;  // guards jobs
    auto consume = [&] {
        for (;;) {
            std::packaged_task<TResult()> job;
            {
                const std::lock_guard lock(mutex);
                if (jobs.empty()) {
                    return;
                }
                job = std::move(jobs.front());
                jobs.pop();
            }
            job();  // stores the result, or the exception, in the job's promise
        }
    };

    // Declared after jobs and mutex: on any exit the consumers are joined
    // (and finish the queue) before what they use is destroyed.
    std::vector<std::jthread> consumers;
    if (num_threads_ == 1) {
        consume();
    } else {
        consumers.reserve(static_cast<std::size_t>(num_threads_));
        for (int t = 0; t < num_threads_; ++t) {
            consumers.emplace_back(consume);
        }
    }
    std::vector<TResult> results;
    results.reserve(inputs.size());
    for (std::future<TResult>& f : futures) {
        results.push_back(f.get());  // waits; rethrows the job's exception
    }
    return results;
}
```

Code

parameters and defaults — `include/nabia/optimization/nongradopt/DifferentialEvolution.h:24-40`
full algorithm — `src/optimization/nongradopt/DifferentialEvolution.cpp:50-193`

7 Risk measures

For portfolio weights x on assets with return covariance Σ and mean μ, write $\sigma(x) = \sqrt{x^\top \Sigma x}$ and take the loss $L = -r^\top x$ with $r \sim \mathcal{N}(\mu, \Sigma)$. Then $L \sim \mathcal{N}(-\mu^\top x, \sigma(x)^2)$. The three measures are:

$$\begin{aligned}
\text{Volatility:} \quad & R(x) = \sigma(x), && \nabla R = \Sigma x/\sigma(x), \\
\text{VaR}_\alpha: \quad & R(x) = -\mu^\top x + z_\alpha\, \sigma(x), && \nabla R = -\mu + z_\alpha \Sigma x/\sigma(x), \\
\text{ES}_\alpha: \quad & R(x) = -\mu^\top x + \frac{\varphi(z_\alpha)}{1-\alpha}\, \sigma(x), && \nabla R = -\mu + \frac{\varphi(z_\alpha)}{1-\alpha} \Sigma x/\sigma(x),
\end{aligned}$$

with $z_\alpha = \Phi^{-1}(\alpha)$.

Proposition 7.1 (Gaussian expected shortfall)**.** *If $L \sim \mathcal{N}(m, s^2)$, then* $\text{ES}_\alpha = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha] = m + s\,\varphi(z_\alpha)/(1-\alpha)$.

Proof. Write $L = m + sZ$. The event is $Z \geq z_\alpha$, which has probability $1-\alpha$. Since $\varphi'(z) = -z\varphi(z)$, we get $\int_{z_\alpha}^{\infty} z\varphi(z)\, dz = \varphi(z_\alpha)$, so $\mathbb{E}[Z \mid Z \geq z_\alpha] = \varphi(z_\alpha)/(1-\alpha)$. □

Proposition 7.2 (Euler decomposition)**.** *Each measure is positively homogeneous of degree one, $R(cx) = cR(x)$ for $c > 0$. Hence $R(x) = \sum_i x_i\, \partial_i R(x)$, and the* risk contributions $\text{RC}_i = x_i \partial_i R$ *add up exactly to the total risk.*

Proof. Differentiate $R(cx) = cR(x)$ with respect to c at $c = 1$. □

At $\sigma = 0$ the gradient of σ is set to zero, which is a valid subgradient of the seminorm. The quantile Φ^{-1} uses Acklam's rational approximation (relative error below $1.2 \cdot 10^{-9}$), refined to full precision by one Halley step on $\Phi(x) - p$:

$$x \leftarrow x - \frac{u}{1 + xu/2}, \qquad u = (\Phi(x) - p)\sqrt{2\pi}\, e^{x^2/2}.$$

Code 15: Volatility gradient, the building block of every measure
`src/risk/IRiskMeasure.cpp:47-55`

```
Eigen::VectorXd IRiskMeasure::portfolio_volatility_gradient(const Eigen::VectorXd& x) const {
    check_dimension(x, "portfolio");
    Eigen::VectorXd sigma_x = covariance_matrix_ * x;
    const double sigma = std::sqrt(std::max(0.0, x.dot(sigma_x)));
    if (sigma == 0.0) {
        return Eigen::VectorXd::Zero(dim());
    }
    return sigma_x / sigma;
}
```

Code 16: Gaussian VaR and its gradient
`src/risk/measures/ValueAtRisk.cpp:27-32`

```
double ValueAtRisk::compute_risk(const Eigen::VectorXd& x) const {
    return -expected_returns_.dot(x) + z_ * portfolio_volatility(x);
}

Eigen::VectorXd ValueAtRisk::compute_marginal_risk_contributions(const Eigen::VectorXd& x) const {
    return -expected_returns_ + z_ * portfolio_volatility_gradient(x);
```

Code

risk measure interface — `include/nabia/risk/IRiskMeasure.h:18-43`
risk contributions $x \odot \nabla R$ — `src/risk/IRiskMeasure.cpp:31-33`
volatility — `src/risk/measures/Volatility.cpp:7-12`
expected shortfall (constant $\varphi(z_\alpha)/(1-\alpha)$) — `src/risk/measures/ExpectedShortfall.cpp:12-33`
normal quantile: Halley refinement — `src/stats/normal.cpp:50-53`

8 Risk budgeting

Given budgets $b_i > 0$ with $\sum_i b_i = 1$, the *risk budgeting portfolio* is the long-only x with $\mathrm{RC}_i(x)/R(x) = b_i$ for all i. Equal budgets give the equal risk contribution portfolio.

Theorem 8.1. *Let R be convex, positively homogeneous of degree one, differentiable on $y > 0$, and positive on $\{y \geq 0,\ y \neq 0\}$. Then*

$$f(y) = R(y) - \sum_i b_i \log y_i$$

has a unique minimizer $y^* > 0$, and $x^* = y^*/\sum_i y_i^*$ is the unique long-only portfolio with $\mathrm{RC}_i(x^*)/R(x^*) = b_i$. Moreover $R(y^*) = 1$.

Proof. Existence. R is continuous and positive on the compact simplex $\Delta = \{y \geq 0, \sum y_i = 1\}$, so $c = \min_\Delta R > 0$. By homogeneity, $R(y) \geq c \sum_i y_i$ for $y \geq 0$. Hence $f(y) \geq \sum_i (c y_i - b_i \log y_i)$. Each term tends to $+\infty$ as $y_i \to 0^+$ or $y_i \to \infty$, so f is coercive on $y > 0$ and attains its minimum.

Uniqueness. $-\sum b_i \log y_i$ is strictly convex and R is convex, so f is strictly convex.

Characterization. At the minimizer, $\partial_i f = \partial_i R(y^*) - b_i/y_i^* = 0$, so $\mathrm{RC}_i(y^*) = y_i^* \partial_i R(y^*) = b_i$. Summing and using Euler gives $R(y^*) = \sum_i b_i = 1$. Let $S = \sum_i y_i^*$. Because $\partial_i R$ is homogeneous of degree zero, $x^* = y^*/S$ has $\mathrm{RC}_i(x^*) = b_i/S$ and $R(x^*) = 1/S$, so $\mathrm{RC}_i/R = b_i$.

Conversely, if x satisfies the budget condition, then $y = x/R(x)$ satisfies $\partial_i f(y) = 0$ and is therefore the unique minimizer. □

For VaR and ES, the condition "$R > 0$ on long-only portfolios" needs $z\,\sigma(x) > \mu^\top x$. The solver checks it at the start and throws otherwise.

Algorithm. The steps follow `src/risk/RiskBudgeting.cpp:45-111`.

1. **Normalize** the budgets. Empty budgets mean equal risk.

2. **Start** at $y_i^0 \propto b_i/\sigma_i$, scaled so that $R(y^0) = 1$. That scale is the magnitude of the solution by the theorem. The start is the inverse-volatility portfolio for equal budgets, which is exact for uncorrelated assets.

3. **Minimize** f with L-BFGS (memory 20). Points with any $y_i \leq 0$ return $+\infty$, so the log barrier needs no explicit constraint.

4. **Match the tolerance to the budget error.** L-BFGS stops on $\|\nabla f\|_\infty$, but the target is the budget error $e = \max_i |\mathrm{RC}_i/R - b_i|$. Near the solution $R \approx 1$ and $\mathrm{RC}_i - b_i = y_i \nabla_i f$, so $e \lesssim \max_i y_i \, \|\nabla f\|_\infty$. The inner tolerance therefore starts at $\tau/\max_i y_i^0$. If $e > \tau$ afterwards, L-BFGS warm-restarts with tolerance $\tau_{\text{in}} \cdot \frac{1}{2}\tau/e$, at most five times.

5. **Return** $x = y/\sum y$, with risk, contributions, budget error and a convergence flag.

Code 17: Objective and gradient, for any risk measure
`src/risk/RiskBudgeting.cpp:65-73`

```cpp
// f(y) = R(y) - b' log(y),  grad f(y) = dR/dy - b ./ y,  y_i grad_i = RC_i(y) - b_i.
// Points with some y_i <= 0 get +inf, which the line search rejects.
const LBFGS::ValueAndGradient objective = [&](const VectorXd& y, VectorXd& grad) {
    if ((y.array() <= 0.0).any()) {
        grad.setZero();
        return std::numeric_limits<double>::infinity();
    }
    grad = measure.compute_marginal_risk_contributions(y) - b.cwiseQuotient(y);
    return measure.compute_risk(y) - b.dot(VectorXd(y.array().log()));
```

Code 18: Tolerance matching on the budget error
`src/risk/RiskBudgeting.cpp:79-99`

```cpp
// L-BFGS stops on ||grad||_inf, but the budget error is ~ ||y .* grad||_inf,
// so start from tolerance / max(y) and, if needed, warm-restart with a
// tighter tolerance until the budget error itself is below `tolerance`.
VectorXd y = y0;
double inner_tol = params.tolerance / y0.maxCoeff();
int iterations = 0;
constexpr int max_restarts = 5;
for (int restart = 0; restart <= max_restarts && iterations < params.max_iter; ++restart) {
    LBFGSParams lbfgs_params{n};
    lbfgs_params.tolerance = inner_tol;
    lbfgs_params.max_iter = params.max_iter - iterations;
    lbfgs_params.memory = params.memory;
    LBFGS solver(std::make_unique<LBFGSParams>(lbfgs_params), objective);
    y = solver.optimize(y);
    iterations += solver.result().iterations;

    const double err = budget_error(y);
    if (err <= params.tolerance || solver.result().status == LBFGSStatus::LineSearchFailed) {
        break;
    }
    inner_tol *= 0.5 * params.tolerance / err;
```

Code

budget normalization — `src/risk/RiskBudgeting.cpp:19-30`
starting point — `src/risk/RiskBudgeting.cpp:34-41`
`RiskBudgeting<Measure>` wrapper — `include/nabia/risk/RiskBudgeting.h:51-69`

9 The walk-forward portfolio engine

`RiskBudgetingPortfolio` backtests a long-only risk budgeting portfolio from a $T \times N$ matrix of simple returns, where row t holds the returns over $(t-1, t]$. Table 1 lists its parameters, which are also the JSON keys of the CLI and the Python API.

Code

parameter struct — `include/nabia/strategy/RiskBudgetingPortfolio.h:31-57`
validation — `src/strategy/RiskBudgetingPortfolio.cpp:110-136`
the engine — `src/strategy/RiskBudgetingPortfolio.cpp:138-267`

9.1 Timing and absence of look-ahead

Rebalances happen at the close of rows $t_k = W - 1 + kf$ for $k = 0, \ldots, \lfloor (T-W)/f \rfloor$. At rebalance t the engine:

1. estimates moments from rows $t - W + 1, \ldots, t$, all known at the close of t;
2. solves for the risk budgeting weights and scales them to the volatility target;
3. trades from the drifted weights to the targets.

Table 1: Portfolio parameters: engine defaults and the defaults of the AI configuration.

Key	Engine	AI	Meaning
`covariance_window`	60	120	rows W per covariance estimate
`covariance_estimator`	sample	exponential	equal or exponential weights
`ewma_halflife`	20	40	half-life h in rows (exponential)
`budgets`	equal	set by policy	risk budgets per asset
`risk_measure`	volatility	volatility	volatility, `value_at_risk`, `expected_shortfall`
`confidence_level`	0.99	–	VaR/ES level in $(0.5, 1)$
`use_expected_returns`	false	–	VaR/ES: window mean instead of 0
`target_volatility`	none	0.10	annualized target σ^*
`max_leverage`	2.0	3.0	cap $L_{\max}$ on $\sum_i w_i$
`rebalance_frequency`	1	21	trade every k rows
`transaction_cost`	0	0.0005	cost κ per unit traded (5 bp)
`drift_weights`	true	true	weights drift between trades
`initial_capital`	1	10^6	starting equity
`periods_per_year`	252	252	P, for annualization
`solver_tolerance`	10^{-8}	–	budget error target τ
`solver_max_iter`	5000	–	L-BFGS iteration budget
`num_threads`	0 (all)	–	threads for the solves

The new weights earn the returns of rows $t+1$ onwards. A test confirms there is no look-ahead: overwriting the data from row 200 on leaves every weight up to row 200 bit-identical.

9.2 Covariance estimation

For the window's returns $r_1, \dots, r_W$ (oldest first) and weights $w_k \geq 0$ with $\sum w_k = 1$:

$$\hat{\mu} = \sum_k w_k r_k, \qquad \hat{\Sigma} = \frac{\sum_k w_k (r_k - \hat{\mu})(r_k - \hat{\mu})^\top}{1 - \sum_k w_k^2}.$$

The weights are either equal ($w_k = 1/W$) or exponential: $w_k \propto \lambda^{W-k}$ with $\lambda = 2^{-1/h}$, so the newest return has age 0 and weight 1 before normalization.

Proposition 9.1. *For i.i.d. returns with covariance Σ, $\mathbb{E}\hat{\Sigma} = \Sigma$. With equal weights the denominator is $(W-1)/W$, which recovers the usual $1/(W-1)$ estimator.*

Proof. Write $r_k - \hat{\mu} = \sum_j (\delta_{kj} - w_j)(r_j - \mu)$. By independence, $\mathbb{E}[(r_k - \hat{\mu})(r_k - \hat{\mu})^\top] = \sum_j (\delta_{kj} - w_j)^2 \Sigma = (1 - 2w_k + \sum_j w_j^2)\Sigma$. Weighting by w_k and summing gives $(1 - \sum_j w_j^2)\Sigma$. □

The quantity $1/\sum_k w_k^2$ is the effective number of observations.

Code 19: Window moments
`src/strategy/RiskBudgetingPortfolio.cpp:35-53`

```
WindowMoments estimate(const RiskBudgetingPortfolioParams& p, const MatrixXd& window) {
    const Index W = window.rows();
    VectorXd w = VectorXd::Constant(W, 1.0 / static_cast<double>(W));
    if (p.covariance_estimator == CovarianceEstimator::Exponential) {
        // Weight lambda^age with lambda = 2^(-1 / halflife); the newest row has age 0.
        const double lambda = std::exp2(-1.0 / p.ewma_halflife);
        for (Index i = 0; i < W; ++i) {
            w(i) = std::pow(lambda, static_cast<double>(W - 1 - i));
        }
        w /= w.sum();
    }
    WindowMoments m;
    m.mean = window.transpose() * w;
    const MatrixXd centred = window.rowwise() - m.mean.transpose();
    // Unbiased weighted covariance: divide by 1 - sum(w^2) (= (W - 1) / W for equal weights).
```

```
    m.covariance = centred.transpose() * w.asDiagonal() * centred / (1.0 - w.squaredNorm());
    m.covariance = 0.5 * (m.covariance + m.covariance.transpose());
    return m;
}
```

9.3 Allocation, budget schedule and volatility targeting

Budget schedule. The budgets in force at rebalance row t are the last *budget change* with first row at or before t. A schedule row dated d takes effect from the first returns row dated on or after d (a binary search). This is how time-varying budgets from a rule or from agents enter one continuous backtest. Without a schedule, the parameter budgets apply.

Volatility targeting. The risk budgeting weights x (summing to 1) are scaled by the leverage

$$L = \min\Big(L_{\max},\ \frac{\sigma^*}{\sqrt{x^\top \hat{\Sigma} x \cdot P}}\Big),$$

so the predicted annualized volatility is σ^* unless the cap binds. Any weight not invested, $1 - \sum_i w_i$, is cash earning zero.

Code 20: Schedule dates to first rows
`include/nabia/strategy/RiskBudgetingPortfolio.h:166-168`

```
for (Eigen::Index s = 0; s < budget_schedule.rows(); ++s) {
    const auto first = std::lower_bound(index.begin(), index.end(), budget_schedule.time(s));
    schedule.push_back({static_cast<Eigen::Index>(first - index.begin()), budget_schedule.values().row(
    ↪ s).transpose()});
```

Code 21: Budgets in force at a rebalance
`src/strategy/RiskBudgetingPortfolio.cpp:164-174`

```
// Budgets in force at a rebalance row: the last change at or before it.
auto budgets_at = [&](Index row) -> const VectorXd& {
    const VectorXd* b = &default_budgets;
    for (const BudgetChange& change : schedule) {
        if (change.first_row > row) {
            break;
        }
        b = &change.budgets;
    }
    return *b;
};
```

Code 22: Target weights of one rebalance date
`src/strategy/RiskBudgetingPortfolio.cpp:92-106`

```
Target solve_target(const RiskBudgetingPortfolioParams& p, const VectorXd& budgets, const MatrixXd&
    ↪ window,
                    const std::optional<VectorXd>& warm_start) {
    const WindowMoments m = estimate(p, window);
    const RiskBudgetingResult rb = allocate(p, budgets, m, warm_start);
    Target target;
    if (p.target_volatility) {
        const double ppy = static_cast<double>(p.periods_per_year);
        const double predicted = std::sqrt(std::max(0.0, rb.weights.dot(m.covariance * rb.weights)) *
    ↪ ppy);
        target.leverage = predicted > 0.0 ? std::min(p.max_leverage, *p.target_volatility / predicted)
    ↪ : p.max_leverage;
    }
    target.weights = target.leverage * rb.weights;
    target.contributions = rb.relative_contributions;
    target.converged = rb.converged;
    return target;
}
```

Code

risk measure selection — `src/strategy/RiskBudgetingPortfolio.cpp:67-79`

9.4 Phase 1: parallel solves

The targets of all rebalance dates depend only on their own windows, so they are solved first, in parallel. Each solve warm-starts from the previous date's unlevered weights, which cuts iterations sharply because consecutive windows differ by a few rows. Warm starts make solves depend on each other, so dates are grouped into *fixed* blocks of 8 consecutive rebalances:

- one block is one task;
- warm starts happen only within a block;
- the block boundaries do not depend on the thread count.

Results are therefore bit-identical for any number of threads. With 50 assets, daily rebalancing and 10 years of data (2,271 solves), a backtest takes 375 ms on one thread and 53 ms on eight.

Code 23: Phase 1

`src/strategy/RiskBudgetingPortfolio.cpp:192-208`

```
// Phase 1 (parallel): target weights of every rebalance date. Each task
// solves one block of consecutive dates, warm-starting within the block.
std::vector<Target> targets(static_cast<std::size_t>(n_rebalances));
const Index n_blocks = (n_rebalances + kWarmStartBlock - 1) / kWarmStartBlock;
const int threads = nabia::detail::resolve_threads(p.num_threads, n_blocks);
nabia::detail::parallel_for(n_blocks, threads, [&](std::int64_t block) {
    std::optional<VectorXd> warm_start;
    const Index first = block * kWarmStartBlock;
    const Index last = std::min(n_rebalances, first + kWarmStartBlock);
    for (Index k = first; k < last; ++k) {
        const Index t = c.rebalance_rows[static_cast<std::size_t>(k)];
        Target target = solve_target(p, budgets_at(t), returns.middleRows(t - W + 1, W), warm_start);
        warm_start = target.weights / target.leverage;  // the unlevered risk budgeting weights
        targets[static_cast<std::size_t>(k)] = std::move(target);
    }
});
c.threads_used = threads;
```

Code 24: Thread pool helper

`src/common/parallel_for.hpp:27-57`

```
void parallel_for(std::int64_t count, int threads, const Task& task) {
    std::atomic<std::int64_t> next{0};
    std::mutex error_mutex;
    std::exception_ptr error;
    std::int64_t error_index = count;

    auto work = [&] {
        for (std::int64_t i = next++; i < count; i = next++) {
            try {
                task(i);
            } catch (...) {
                const std::lock_guard lock(error_mutex);
                if (i < error_index) {
                    error_index = i;
                    error = std::current_exception();
                }
            }
        }
    };
    {
        std::vector<std::jthread> workers;
        workers.reserve(static_cast<std::size_t>(std::max(0, threads - 1)));
        for (int t = 1; t < threads; ++t) {
```

```
            workers.emplace_back(work);
        }
        work();
    }  // joining the workers makes all their writes visible here
    if (error) {
        std::rethrow_exception(error);
    }
}
```

Code

thread count (0 = hardware concurrency, capped by tasks) — `src/common/parallel_for.hpp:16-20`

9.5 Phase 2: drift, trading and accounting

Phase 2 walks through time. With held weights w_t and asset returns r_t, each row does the following:

$$
\begin{aligned}
&g_t = w_t^\top r_t && \text{gross return,}\\
&w \leftarrow w_t \odot (1 + r_t)/(1 + g_t) && \text{drift,}\\
&\tau_t = \|w^{\text{target}} - w\|_1, \quad c_t = \kappa\,\tau_t && \text{turnover and cost (rebalance rows),}\\
&n_t = (1 + g_t)(1 - c_t) - 1, \quad E_t = E_{t-1}(1 + n_t) && \text{net return and equity.}
\end{aligned}
$$

Drift. Holding i is worth $w_i E$ and grows to $w_i E(1 + r_i)$. Cash earns nothing, so equity grows to $E(1 + g)$. The new weight is the ratio of the two.

Costs. They are paid out of equity at the close, after the period's return, which gives $E(1 + g)(1 - c)$. The first trade buys the whole portfolio from cash, so its turnover is $\sum_i w_i$.

Checks. The engine stops with an error if $1 + g_t \le 0$ (the portfolio lost all its capital). These identities are checked to 10^{-15} in the tests.

Code 25: Phase 2

`src/strategy/RiskBudgetingPortfolio.cpp:210-254`

```
// Phase 2 (sequential): drift, trade and account through time.
VectorXd w = VectorXd::Zero(N);  // current weights (fractions of equity)
double equity = p.initial_capital;
const double ppy = static_cast<double>(p.periods_per_year);
Index next = 0;  // next rebalance

for (Index t = 0; t < T; ++t) {
    // 1. Earn this period's returns with the weights set at the previous close.
    c.weights.row(t) = w.transpose();
    const VectorXd r = returns.row(t).transpose();
    const double gross = w.dot(r);
    if (!(1.0 + gross > 0.0)) {
        throw std::invalid_argument("RiskBudgetingPortfolio: the portfolio lost all its capital at row
    ↪ " +
                                    std::to_string(t));
    }
    if (p.drift_weights) {
        // Each holding grows by (1 + r_i), equity by (1 + gross); cash is the remainder.
        w = w.cwiseProduct((VectorXd::Ones(N) + r)) / (1.0 + gross);
    }

    // 2. Rebalance at the close, with the data known at t.
    double cost = 0.0;
    if (next < n_rebalances && c.rebalance_rows[static_cast<std::size_t>(next)] == t) {
        const Target& target = targets[static_cast<std::size_t>(next)];
        const double traded = (target.weights - w).lpNorm<1>();
        cost = p.transaction_cost * traded;
        w = target.weights;
```

```
        c.target_weights.row(next) = target.weights.transpose();
        c.risk_contributions.row(next) = target.contributions.transpose();
        c.leverage(next) = target.leverage;
        c.all_converged = c.all_converged && target.converged;
        c.turnover(t) = traded;
        ++next;
    }

    // 3. Accounting: costs are paid out of the equity at the close.
    const double net = (1.0 + gross) * (1.0 - cost) - 1.0;
    const double new_equity = equity * (1.0 + net);
    c.gross_returns(t) = gross;
    c.costs(t) = cost;
    c.net_returns(t) = net;
    c.pnl(t) = new_equity - equity;
    c.equity(t) = new_equity;
    equity = new_equity;
```

10 Performance metrics

The metrics use the net returns $n_1, \ldots, n_T$ from the first trade on, with P periods per year and equity E starting at 1.

Metric	Definition
Total return	$\prod_t (1 + n_t) - 1$
Annualized return (CAGR)	$(1 + \text{total})^{P/T} - 1$
Annualized volatility	$\text{sd}(n)\sqrt{P}$ (sample standard deviation)
Sharpe ratio	$\bar{n}\,P/(\text{sd}(n)\sqrt{P})$ (zero risk-free rate)
Sortino ratio	$\bar{n}\,P/\big(\sqrt{\frac{1}{T}\sum_t \min(n_t, 0)^2}\,\sqrt{P}\big)$
Maximum drawdown	$\max_t \big(1 - E_t / \max_{s \le t} E_s\big)$
Max drawdown duration	longest run of periods below a previous peak
Calmar ratio	annualized return / maximum drawdown
Hit rate	fraction of periods with $n_t > 0$
Best / worst period	$\max_t n_t$, $\min_t n_t$
Turnover	average τ per rebalance; annualized $\sum_t \tau_t\,P/T$
Total costs	$\sum_t c_t$

Code 26: Performance metrics
`src/strategy/PerformanceMetrics.cpp:31-67`

```
PerformanceMetrics compute_performance(const Eigen::VectorXd& returns, int periods_per_year) {
    if (returns.size() == 0 || !returns.allFinite()) {
        throw std::invalid_argument("compute_performance: returns must be non-empty and finite");
    }
    if (periods_per_year <= 0) {
        throw std::invalid_argument("compute_performance: periods_per_year must be positive");
    }
    const auto n = static_cast<double>(returns.size());
    const double ppy = static_cast<double>(periods_per_year);
    PerformanceMetrics m;
    m.periods = returns.size();

    const Eigen::VectorXd equity = equity_curve(returns, 1.0);
    m.total_return = equity(equity.size() - 1) - 1.0;
    m.annualized_return = m.total_return > -1.0 ? std::pow(1.0 + m.total_return, ppy / n) - 1.0 : -1.0;

    const double mean = returns.mean();
    const double var = returns.size() > 1 ? (returns.array() - mean).square().sum() / (n - 1.0) : 0.0;
    m.annualized_volatility = std::sqrt(var * ppy);
    m.sharpe_ratio = m.annualized_volatility > 0.0 ? mean * ppy / m.annualized_volatility : 0.0;
    const double downside = std::sqrt(returns.cwiseMin(0.0).squaredNorm() / n) * std::sqrt(ppy);
    m.sortino_ratio = downside > 0.0 ? mean * ppy / downside : 0.0;
```

```
    const Eigen::VectorXd dd = drawdown_series(equity, 1.0);
    m.max_drawdown = dd.maxCoeff();
    Eigen::Index run = 0;
    for (Eigen::Index t = 0; t < dd.size(); ++t) {
        run = dd(t) > 0.0 ? run + 1 : 0;
        m.max_drawdown_duration = std::max(m.max_drawdown_duration, run);
    }
    m.calmar_ratio = m.max_drawdown > 0.0 ? m.annualized_return / m.max_drawdown : 0.0;

    m.hit_rate = static_cast<double>((returns.array() > 0.0).count()) / n;
    m.best_period = returns.maxCoeff();
    m.worst_period = returns.minCoeff();
    return m;
}
```

Code

drawdown series — `src/strategy/PerformanceMetrics.cpp:21-29`

Part III
Applications, bindings and data

11 Command-line application

`nabia_rb_backtest --prices prices.csv --config config.json --output out.csv`:

- **Prices** come from a CSV with ISO dates and one column per asset. Empty cells are missing values.
- **Alignment**: dates where any asset is missing are dropped, and the drop is reported.
- **Returns**: simple returns are computed after alignment.
- **Parameters** come from a JSON file with the keys of Table 1, plus an optional `start` and `end`. Unknown keys are rejected, so a typo fails loudly. Budgets can be a list or an object keyed by asset name.
- **Output**: a CSV row per date with the weights, the risk contributions on rebalance dates, turnover, costs, returns, PnL, equity and drawdown, and an optional metrics CSV.

Code

entry point — `apps/rb_backtest/main.cpp:94-139`
JSON parser — `apps/rb_backtest/config.cpp:68-163`
CSV reader — `apps/rb_backtest/csv_io.cpp:124-182`
CSV writer — `apps/rb_backtest/csv_io.cpp:184-227`

12 Python bindings and pandas API

The Python module reuses the CLI's pipeline and JSON parser, so both accept the same parameters with the same error messages. Data crosses the boundary in simple forms:

- dates as `int64` days since 1970-01-01 (numpy `datetime64[D]`);
- prices as a `float64` matrix;
- parameters as JSON text.

The GIL is released during the computation, so the engine's threads run in parallel.

Two functions are exposed:

- `nabia.backtest(prices, params, budget_schedule=None)` returns a result with pandas frames and metrics;
- `nabia.latest_weights(prices, params)` solves only the last rebalance. It is used by the agents to turn budgets into target weights.

Code 27: Shared preparation: parse, align, window, returns, budgets
`python/src/nabia_module.cpp:65-96`

```
Prepared prepare(const DaysArray& days, const std::vector<std::string>& assets, const MatrixXd& prices,
                 const std::string& config_json) {
    if (prices.rows() != days.size() || prices.cols() != static_cast<Index>(assets.size())) {
        throw std::invalid_argument("prices must be (dates x assets): got " + std::to_string(prices.
    ↪ rows()) + " x " +
                                    std::to_string(prices.cols()) + " for " + std::to_string(days.size
    ↪ ()) +
                                    " dates and " + std::to_string(assets.size()) + " assets");
    }
    Prepared p;
    p.config = nabia::app::parse_config(config_json);
    DataFrame<Date> px(to_dates(days), assets, prices);
    const Index read = px.rows();
    px = px.drop_missing();
    p.dropped_dates = read - px.rows();
    if (px.rows() >= 2 && (p.config.start || p.config.end)) {
        px = px.between(p.config.start.value_or(px.time(0)), p.config.end.value_or(px.time(px.rows() -
    ↪ 1)));
    }
    if (px.rows() < 2) {
        throw std::invalid_argument("need at least 2 dates with a price for every asset");
    }
    for (Index t = 0; t < px.rows(); ++t) {
        for (Index j = 0; j < px.cols(); ++j) {
            if (!(px.values()(t, j) > 0.0)) {
                throw std::invalid_argument("prices must be positive for returns: " + px.columns()[
    ↪ static_cast<std::size_t>(j)] +
                                            " is " + std::to_string(px.values()(t, j)) + " on " +
                                            nabia::app::format_date(px.time(t)));
            }
        }
    }
    p.returns = px.pct_change();
    p.config.params.budgets = nabia::app::resolve_budgets(p.config, assets);
    return p;
}
```

Code 28: pandas to C++
`python/nabia/portfolio.py:104-110`

```
def _inputs(prices: pd.DataFrame, params: Mapping[str, Any] | None) -> tuple[np.ndarray, list[str], np.
    ↪ ndarray, str]:
    if not isinstance(prices, pd.DataFrame):
        raise TypeError("prices must be a pandas DataFrame (dates x assets)")
    if prices.shape[1] == 0:
        raise ValueError("prices has no columns")
    values = np.ascontiguousarray(prices.to_numpy(dtype=np.float64, na_value=np.nan))
    return _to_days(prices.index), [str(c) for c in prices.columns], values, _config_json(params)
```

Code

`backtest` binding — `python/src/nabia_module.cpp:119-160`
`latest_weights` binding — `python/src/nabia_module.cpp:162-189`
module definition — `python/src/nabia_module.cpp:193-206`
Python `backtest` — `python/nabia/portfolio.py:113-183`
Python `latest_weights` — `python/nabia/portfolio.py:186-203`

13 Market data

13.1 Universe and sources

The default download universe has four asset classes:

- equity indices: S&P 500, Nasdaq 100, Russell 2000, Euro Stoxx 50, DAX, FTSE 100, Nikkei 225, Hang Seng;
- US Treasury futures: 2Y, 5Y, 10Y, Bond, Ultra Bond;
- commodity futures: WTI, Brent, natural gas, gold, silver, copper, corn, wheat, soybeans;
- crypto against USD: BTC, ETH, SOL, XRP, ADA, LTC.

The AI portfolio uses the nine assets of Table 2.

Table 2: Default AI portfolio universe.

Symbol	Asset	Class
`^GSPC`	S&P 500	equity index
`^STOXX50E`	Euro Stoxx 50	equity index
`^N225`	Nikkei 225	equity index
`ZN=F`	US 10-Year T-Note future	bond future
`ZB=F`	US Treasury Bond future	bond future
`BZ=F`	Brent crude oil future	commodity
`GC=F`	Gold future	commodity
`HG=F`	Copper future	commodity
`BTC/USD`	Bitcoin	crypto

Sources.

- **Yahoo Finance** (`yfinance`) provides indices and futures, in batches. Futures are front-month continuous series, which jump at contract rolls. WTI settled negative on 2020-04-20, where simple returns are undefined, so the portfolio uses Brent.
- **Crypto exchanges** (`ccxt`, Coinbase by default) provide daily candles from public data, with no key. History is paged in windows of 300 candles. Windows before a coin's listing date are empty and are stepped over. Paging stops if the exchange ignores the start date.

Code 29: Crypto daily candles with pagination
`python/nabia/marketdata/sources.py:147-188`

```python
def fetch(self, symbols: Sequence[str], start: dt.date, end: dt.date | None = None) -> pd.DataFrame:
    client = self._exchange()
    start_ms = int(dt.datetime.combine(start, dt.time(), dt.UTC).timestamp() * 1000)
    last_day = end or dt.datetime.now(dt.UTC).date()
    stop_ms = int(dt.datetime.combine(last_day + dt.timedelta(days=1), dt.time(), dt.UTC).timestamp() *
    ↪  1000)
    frames = []
    for symbol in symbols:
        if symbol not in client.markets:
            log.warning("%s is not listed on %s", symbol, getattr(client, "id", "the exchange"))
            continue
        candles: dict[int, list[float]] = {}
        since = start_ms
        while since < stop_ms:
            page = self._page(client, symbol, since)
            if not page:
                # Nothing in this window: before the listing date (or a gap). Step
```

```
                # forward a page; stop once past what has been collected so far.
                if candles:
                    break
                since += self.page_size * _DAY_MS
                continue
            for candle in page:
                if start_ms <= candle[0] < stop_ms:
                    candles[int(candle[0])] = candle
            next_since = int(page[-1][0]) + _DAY_MS
            if next_since <= since:  # no progress: the exchange ignores 'since'
                break
            since = next_since
        if not candles:
            continue
        rows = [candles[k] for k in sorted(candles)]
        frames.append(pd.DataFrame({
            "symbol": symbol,
            "date": [dt.datetime.fromtimestamp(r[0] / 1000, dt.UTC).date() for r in rows],
            "open": [r[1] for r in rows],
            "high": [r[2] for r in rows],
            "low": [r[3] for r in rows],
            "close": [r[4] for r in rows],
            "adj_close": [r[4] for r in rows],
            "volume": [r[5] for r in rows],
        }).astype({c: "float64" for c in PRICE_COLUMNS[2:]}))
    return pd.concat(frames, ignore_index=True)[PRICE_COLUMNS] if frames else empty_prices()
```

13.2 Storage and incremental updates

Prices are stored per (symbol, date): open, high, low, close, adjusted close and volume.

- **Upserts.** Writes use `INSERT ... ON CONFLICT DO UPDATE`, in the SQLite or PostgreSQL dialect, so re-downloading is idempotent and picks up revisions.
- **Incremental updates.** A stored symbol is re-fetched from five days before its last date, which corrects recent revisions.
- **Batching.** Symbols are grouped by source and start date.

Code 30: Portable upsert
python/nabia/marketdata/store.py:66-81

```
def upsert_rows(engine: Engine, table: Table, rows: list[dict], keys: Sequence[str], chunk: int = 500)
    ↪ -> None:
    """Insert rows, updating the non-key columns of rows whose keys already exist."""
    if not rows:
        return
    dialect = engine.dialect.name
    if dialect == "sqlite":
        from sqlalchemy.dialects.sqlite import insert
    elif dialect == "postgresql":
        from sqlalchemy.dialects.postgresql import insert
    else:  # pragma: no cover - other databases are not supported yet
        raise NotImplementedError(f"upsert is not implemented for {dialect}")
    with engine.begin() as connection:
        for i in range(0, len(rows), chunk):
            statement = insert(table).values(rows[i : i + chunk])
            updates = {c.name: statement.excluded[c.name] for c in table.columns if c.name not in keys}
            connection.execute(statement.on_conflict_do_update(index_elements=list(keys), set_=updates)
    ↪ )
```

Code 31: Incremental download
python/nabia/marketdata/download.py:19-72

```
def update_market_data(
    store: MarketDataStore,
    source: MarketDataSource | Mapping[str, MarketDataSource],
```

```
    assets: Sequence[Asset],
    start: dt.date,
    end: dt.date | None = None,
    full: bool = False,
    overlap_days: int = DEFAULT_OVERLAP_DAYS,
) -> dict[str, int]:
    """Downloads daily prices for ``assets`` and upserts them into ``store``.

    ``source`` is either one source for every asset, or a mapping from source
    name to source, in which case each asset is fetched from ``asset.source``
    (e.g. ``{"yahoo": YahooFinanceSource(), "crypto": CryptoSource()}``).

    Incremental by default: a symbol already in the store is fetched from
    ``overlap_days`` before its last stored date (so recent revisions are
    picked up); a new symbol from ``start``. ``full=True`` re-downloads
    everything from ``start``. Returns the number of rows written per symbol
    (0 for symbols the source returned no data for).
    """
    sources: dict[str, MarketDataSource]
    if isinstance(source, Mapping):
        sources = dict(source)
        unknown = {a.source for a in assets} - sources.keys()
        if unknown:
            raise ValueError(f"no source given for: {sorted(unknown)}")
        source_of = {a.symbol: a.source for a in assets}
    else:
        sources = {source.name: source}
        source_of = {a.symbol: source.name for a in assets}
    for name in sorted({source_of[a.symbol] for a in assets}):
        store.upsert_assets([a for a in assets if source_of[a.symbol] == name], source=name)
    last = {} if full else store.last_dates()

    # Group symbols by source and fetch start, so each group is one batched request.
    groups: dict[tuple[str, dt.date], list[str]] = {}
    for asset in assets:
        from_date = start
        if asset.symbol in last:
            from_date = max(start, last[asset.symbol] - dt.timedelta(days=overlap_days))
        groups.setdefault((source_of[asset.symbol], from_date), []).append(asset.symbol)

    written = {asset.symbol: 0 for asset in assets}
    for (name, from_date), symbols in sorted(groups.items()):
        log.info("fetching %d symbols from %s since %s", len(symbols), name, from_date)
        prices = sources[name].fetch(symbols, from_date, end)
        for symbol, rows in prices.groupby("symbol"):
            written[str(symbol)] = store.upsert_prices(rows)
            log.info("%s: %d prices stored (%s to %s)", symbol, written[str(symbol)], rows["date"].min
    ↪ (), rows["date"].max())
    for symbol, count in written.items():
        if count == 0:
            log.warning("no data for %s", symbol)
    return written
```

Code

universe — `python/nabia/marketdata/universe.py:24-70`
Yahoo source — `python/nabia/marketdata/sources.py:48-67`
loading a price matrix — `python/nabia/marketdata/store.py:136-168`

Part IV
The AI layer

14 Agents: roles, data flow and configuration

Agent	Tools	Output
News analyzer	`news` server; on Claude also web search and web fetch	stored articles with topic, assets, sentiment, relevance
Stats analyzer	`stats` server	a structured report (`StatsReport`) with a budget tilt per asset
Execution agent	`news` and `stats` read-only, `portfolio`	risk budgets with a rationale, then target weights from the C++ engine

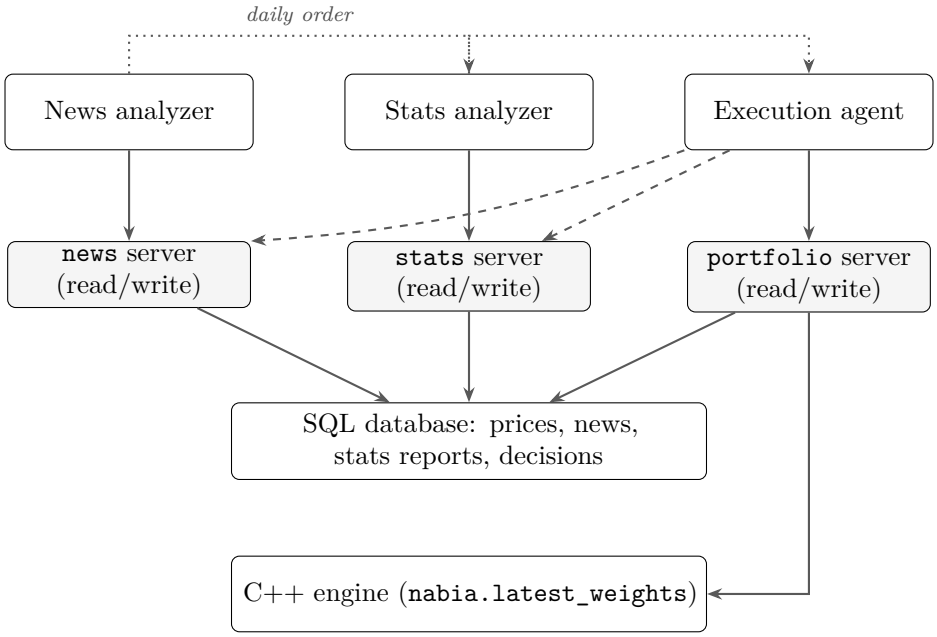


Figure 2: The three agents, the MCP servers they use, and their order in the daily pipeline. Dashed arrows: read-only access (the servers run with `--read-only`).

Pipelines.

- The *daily* pipeline runs news, then stats, then execution, for today.
- The *backtest* pipeline runs stats and execution at each decision date (§19.4).

Every agent run is stored with its turns, tool calls (inputs and outputs, truncated to 2,000 characters), token usage and any error, which gives a complete audit trail.

Code

running and recording an agent — `python/nabia/ai/agents/team.py:111-125`
daily pipeline — `python/nabia/ai/pipeline.py:36-50`
database tables — `python/nabia/ai/db.py:39-120`

14.1 Configuration

Agents, servers and pipeline share one `AIConfig`, saved as JSON so that the MCP server processes read the same settings. Unknown keys are rejected. Validation requires at least two distinct symbols, $0 < b_{\min} \leq 1/n \leq b_{\max} \leq 1$, a known provider, and no `budgets` in `portfolio`, because budgets are what the agents decide.

Field	Default	Meaning
`db_url`	`'sqlite:///market.db'`	Database with market data (nabia.marketdata), news, stats reports and decisions.
`symbols`	the 9 symbols of Table 2	Portfolio universe: one risk budget per symbol.
`portfolio`	Table 1	nabia.backtest parameters (without budgets, which the execution agent sets).
`min_budget`	`0.02`	Lower bound on each normalized risk budget, enforced by set_risk_budgets.
`max_budget`	`0.4`	Upper bound on each normalized risk budget, enforced by set_risk_budgets.
`provider`	`'anthropic'`	Model provider: "anthropic" (Claude API) or "ollama" (a local model).
`model`	`'claude-opus-5-5'`	Claude model, when provider is "anthropic".
`effort`	`{'news': 'medium', 'stats': 'high', '...`	Claude effort level per agent (news, stats, execution).
`ollama_model`	`'qwen3.8:latest'`	Local model, when provider is "ollama" (it must support tool calling: `ollama show <model>`).
`ollama_host`	`'http://localhost:11434'`	URL of the Ollama server.
`ollama_options`	`{'temperature': 0.2, 'num_ctx': 32768}`	Ollama model options; num_ctx must hold the prompts, tool schemas and tool results.
`ollama_think`	`False`	Let thinking models reason before answering (slower, sometimes better tool use).
`news_feeds`	10 feeds, §20.1	RSS / Atom feeds read by fetch_news_feeds, by source name.
`news_max_searches`	`10`	Cap on web searches per news collection run.
`news_lookback_days`	`7`	Window of stored news the execution agent reviews.
`stats_history_days`	`400`	Calendar days of prices loaded by the stats tools (covers 1y signals).

Code

`AIConfig` — `python/nabia/ai/config.py:40-104`

15 MCP servers and tool catalogue

15.1 Design

Each server is a standalone MCP server over stdio (`python -m nabia.ai.servers.<name> --config ... --run ... [--read-only]`). The agents use them, and so can any MCP client, for example Claude Code. A server is built in three layers:

- A `ServerContext` holds the configuration, the run name and the two stores (market data and agent data).
- Tools are plain Python functions registered with `@tool(server)`.
- The MCP library derives each tool's *description* from the function's docstring and its *input schema* from the type annotations. The catalogue below is extracted from the same source, so it is exactly what the model sees.

Four rules apply to every tool.

Point in time Analytical tools take `as_of` (`YYYY-MM-DD`) and read only data dated on or before it. A test rewrites every price after `as_of` and checks that every tool's output is unchanged.

Errors the model can act on Anticipated failures, such as wrong symbols or out-of-range values, raise `ValueError` or `KeyError`. The decorator turns them into MCP `ToolError`s carrying the message, so the model sees why a call failed and can correct it. Any other exception would reach the model only as a generic error.

Read-only mode `--read-only` omits the tools that write. The execution agent cannot store news or overwrite statistics reports.

Runs `--run <name>` scopes reports and decisions to a backtest run, or to `live`, so backtests never mix with live decisions.

Code 32: Tool registration
`python/nabia/ai/servers/common.py:94-114`

```python
def tool(server: MCPServer) -> Callable[[F], F]:
    """Registers a function as an MCP tool, reporting ValueError / KeyError to the
    model as tool errors with their message (MCP otherwise treats them as crashes
    and hides the text), so the agent can correct its call."""

    def decorator(fn: F) -> F:
        @functools.wraps(fn)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            try:
                return fn(*args, **kwargs)
            except (ValueError, KeyError) as error:
                raise ToolError(str(error)) from error

        # Resolved annotations, so schema generation does not depend on this module's globals.
        wrapper.__signature__ = inspect.signature(fn, eval_str=True)  # type: ignore[attr-defined]
        wrapper.__annotations__ = {**{k: p.annotation for k, p in wrapper.__signature__.parameters.
    ↪ items()},
                                   "return": wrapper.__signature__.return_annotation}
        server.tool()(wrapper)
        return fn

    return decorator
```

Code 33: The as-of date
`python/nabia/ai/servers/common.py:52-60`

```python
def parse_as_of(text: str) -> dt.date:
    """'YYYY-MM-DD' (or 'today'). The as-of date bounds every query: tools never
    return data dated after it, which keeps backtests free of look-ahead."""
    if text == "today":
        return dt.date.today()
    try:
        return dt.date.fromisoformat(text)
    except ValueError as error:
        raise ValueError(f"as_of must be YYYY-MM-DD, got {text!r}") from error
```

Code

`ServerContext` — `python/nabia/ai/servers/common.py:23-49`
prices up to as_of, universe check — `python/nabia/ai/servers/common.py:38-49`
JSON cleaning (NaN to null, rounding) — `python/nabia/ai/servers/common.py:63-79`
news server — `python/nabia/ai/servers/news.py:42-136`
stats server — `python/nabia/ai/servers/stats.py:31-105`
portfolio server — `python/nabia/ai/servers/portfolio.py:16-72`

15.2 Catalogue

The rest of this section is generated from the server sources. For each tool it gives the description shown to the model, whether the tool writes, the source location, and the parameters with their

types and defaults. The input schemas of the structured tools follow each server. In the news schemas, `Topic` is one of:

- monetary_policy, inflation, growth, labor, earnings, fiscal, geopolitics;
- energy, metals, agriculture, crypto, rates, credit, fx, regulation, other.

`AssetClass` is one of equity_index, bond_future, commodity and crypto.

15.3 Server `nabia-news`

Module: `nabia.ai.servers.news` (`python/nabia/ai/servers/news.py`). **Instructions sent to the client:** *Financial news database of the nabia trend portfolio. Dates are YYYY-MM-DD; queries never return articles published after as_of.*

`store_news_articles`

writes; removed in read-only mode `python/nabia/ai/servers/news.py:50-68`

Store news articles (upsert by URL, so storing an article twice updates it).

Only store articles with a real URL and publication date you have seen. Symbols must be portfolio symbols (see the news_summary universe).

`articles list[NewsArticle]`, required

`fetch_news_feeds`

writes; removed in read-only mode `python/nabia/ai/servers/news.py:71-90`

Latest headlines from the configured financial news RSS feeds (markets, economy, central banks, energy, crypto), newest first, skipping articles already stored. Use it to find candidate stories to classify and store.

`max_age_days int`, default 2

`max_items int`, default 60

`query_news`

read-only `python/nabia/ai/servers/news.py:92-99`

Stored articles published in the `days` days up to and including as_of, most recent and relevant first, optionally filtered by topic, symbol or asset class.

`as_of str`, required

`days int`, default 7

`topic Topic | None`, default None

`symbol str | None`, default None

`asset_class AssetClass | None`, default None

`limit int`, default 30

`news_summary`

read-only `python/nabia/ai/servers/news.py:101-134`

Aggregate view of the news in the `days` days up to as_of: article counts and relevance-weighted sentiment per topic, asset class and symbol, plus the top headlines.

`as_of str`, required

`days int`, default 7

Input schema `NewsArticle`

pydantic model `python/nabia/ai/servers/news.py:27-39`

One news item, as extracted by the news analyzer.

`url str`. Canonical URL of the article (the unique key)

`published_date dt.date`. Publication date, YYYY-MM-DD

`title str`

`source str`. Publisher, e.g. Reuters, Bloomberg, FT

`topic Topic`. Main topic

`asset_classes list[AssetClass]`. Asset classes the news affects

`symbols list[str]`. Portfolio symbols directly affected

`sentiment float`. $\geq$ -1.0; $\leq$ 1.0; Price impact on the affected assets: -1 bearish .. +1 bullish

`relevance float`. $\geq$ 0.0; $\leq$ 1.0; Importance for a cross-asset portfolio: 0 .. 1

`summary str`. Two or three factual sentences: what happened and why it matters

15.4 Server `nabia-stats`

Module: `nabia.ai.servers.stats` (`python/nabia/ai/servers/stats.py`). **Instructions sent to the client:** *Statistics of the nabia portfolio assets. Every tool takes as_of (YYYY-MM-DD) and uses only prices dated on or before it. Volatilities are annualized; returns are simple.*

`list_universe`

read-only `python/nabia/ai/servers/stats.py:38-55`

Portfolio symbols with their asset class, name and price coverage up to as_of.

`as_of str`, required

`correlation_matrix`

read-only `python/nabia/ai/servers/stats.py:57-64`

Correlation of daily returns over the last `window` dates where all assets traded, plus the average pairwise correlation.

`as_of str`, required

`window int`, default 60

`rolling_volatility`

read-only `python/nabia/ai/servers/stats.py:66-71`

Annualized volatility per asset over 20, 60 and 120 days, and the percentile of today's 20-day volatility within its last year (1.0 = highest of the year).

`as_of str`, required

`trend_signals`

read-only `python/nabia/ai/servers/stats.py:73-78`

Per asset: 1/3/6/12-month returns, distance from the 50/200-day moving averages, volatility-scaled 12-1 month momentum, current and 1-year max drawdown.

`as_of str`, required

`market_regime`

read-only `python/nabia/ai/servers/stats.py:80-85`

Cross-asset regime: average correlation now vs over a year, median volatility percentile, and the share of assets in an uptrend.

`as_of` `str`, required

`get_stats_report`

read-only `python/nabia/ai/servers/stats.py:87-91`

The latest stats report saved on or before as_of (null if none).

`as_of` `str`, required

`save_stats_report`

writes; removed in read-only mode `python/nabia/ai/servers/stats.py:94-103`

Save the structured analysis for as_of (replaces an earlier one for the same date). Include every portfolio symbol in `assets`.

`as_of` `str`, required

`report` `StatsReport`, required

Input schema `AssetView`

pydantic model `python/nabia/ai/servers/stats.py:15-21`

`symbol` `str`

`trend` `str`. e.g. strong up, up, flat, down, strong down

`volatility_regime` `str`. e.g. calm, normal, elevated, stressed

`diversification` `str`. How it correlates with the rest of the portfolio

`budget_tilt` `float`. $\geq$ -1.0; $\leq$ 1.0; Suggested tilt of its risk budget: -1 cut .. +1 raise

`reason` `str`

Input schema `StatsReport`

pydantic model `python/nabia/ai/servers/stats.py:24-28`

`summary` `str`. Cross-asset picture in a few sentences

`regime` `str`. e.g. risk-on trending, risk-off, high-correlation stress, choppy

`assets` `list[AssetView]`

`key_risks` `list[str]`

15.5 Server `nabia-portfolio`

Module: `nabia.ai.servers.portfolio` (`python/nabia/ai/servers/portfolio.py`). **Instructions sent to the client:** *Risk budgets and target weights of the nabia risk budgeting portfolio. Budgets are each asset's share of portfolio risk; they are normalized to sum to 1.*

`get_portfolio_config`

read-only `python/nabia/ai/servers/portfolio.py:23-29`

Universe, budget bounds and the portfolio engine settings (covariance window, rebalancing, volatility target, costs). *No parameters.*

get_previous_decision

read-only python/nabia/ai/servers/portfolio.py:31-35

The most recent risk budget decision strictly before as_of (null if none), with its rationale and resulting weights.

as_of `str`, required

preview_budgets

read-only python/nabia/ai/servers/portfolio.py:37-42

Show how budgets would be normalized and bounded, without saving anything.

budgets `dict[str, float]`, required

set_risk_budgets

writes; removed in read-only mode python/nabia/ai/servers/portfolio.py:45-58

Decide the risk budgets for as_of: one positive number per portfolio symbol (relative sizes; normalized to sum to 1 and bounded to [min_budget, max_budget]). The rationale should cite the evidence (stats, news) behind each tilt.

as_of `str`, required

budgets `dict[str, float]`, required

rationale `str`, required

compute_target_weights

writes; removed in read-only mode python/nabia/ai/servers/portfolio.py:60-70

Target portfolio weights from the budgets decided for as_of: risk budgeting on the trailing covariance (C++ engine), scaled to the volatility target. Call it after set_risk_budgets.

as_of `str`, required

16 The agent runtime

16.1 Starting an agent

`AgentRuntime.run` starts the agent's MCP servers as subprocesses. They use the same Python interpreter and module path, with `--config` pointing to the saved configuration and `--run` set to the run name. The runtime then:

- initializes a client session per server;
- lists the tools of every session and rejects name collisions between servers;
- adds the hosted tools, if any (web search and fetch on Claude);
- runs the loop.

Errors raised inside the loop are re-raised only after the sessions close cleanly. Otherwise the MCP task groups would wrap them in an `ExceptionGroup`.

Code

run: servers, tools, loop — `python/nabia/ai/agents/runtime.py:160-190`
server subprocess parameters — `python/nabia/ai/agents/runtime.py:151-158`
client selection by provider — `python/nabia/ai/agents/runtime.py:133-145`

16.2 The loop

The runtime uses a manual tool-use loop, not an SDK tool runner. One agent talks to several MCP processes at once, mixes their tools with hosted tools, and records every call. Each turn proceeds as follows.

1. **Request.** One request carries the system prompt, the tools and the conversation so far, plus these options:
   - `max_tokens`: 16,000;
   - adaptive thinking;
   - the agent's effort level;
   - prompt caching of the stable prefix (tools, system prompt, history);
   - server-side refusal fallbacks.
2. **Streaming.** The reply is *streamed*. Text and thinking deltas go to the log as they arrive, so a job's log follows the model live (§21).
3. **Stop reasons.**
   - `refusal` raises an error naming the category.
   - `pause_turn` (a long hosted-tool turn) resends the conversation so the model can resume.
   - `max_tokens` is an error.
   - Any other reason that is not `tool_use` ends the run with the final text.
4. **Tools.** All `tool_use` blocks of the turn run concurrently. Their results return *in one user message*. A failing tool, or a crashed server, becomes an error result, so the agent can recover.

A run is limited to 40 turns.

Code 34: The agent loop
`python/nabia/ai/agents/runtime.py:192-238`

```
async def _loop(self, record: AgentRun, system: str, prompt: str, tools: list[dict[str, Any]],
                sessions: dict[str, ClientSession], effort: str) -> None:
    messages: list[dict[str, Any]] = [{"role": "user", "content": prompt}]
    for turn in range(1, self.max_turns + 1):
        log.info("%s: turn %d, waiting for the model...", record.agent, turn)
        started = time.monotonic()
        live = LiveOutput(record.agent)
        response = await self._create(live, dict(
            model=self.config.model,
            max_tokens=MAX_TOKENS,
            system=system,
            tools=tools,
            messages=messages,
            thinking={"type": "adaptive"},
            output_config={"effort": effort},
            cache_control={"type": "ephemeral"},  # caches the stable prefix: tools, system, history
            betas=["server-side-fallback-2026-07-01"],
            fallbacks="default",  # re-run a refused request on a fallback model
        ))
        record.turns += 1
        self._add_usage(record, response.usage)
        log.info("%s: turn %d -> %s (%s in / %s out tokens, %.1fs)", record.agent, turn, response.
    ↪ stop_reason,
                 getattr(response.usage, "input_tokens", "?"), getattr(response.usage, "output_tokens",
    ↪  "?"),
                 time.monotonic() - started)
        for block in response.content:
```

```
            if block.type == "text" and block.text.strip() and not live.chars and response.stop_reason
    ↪ == "tool_use":
                log.info("%s says: %s", record.agent, _preview(block.text, 300))  # not streamed
            elif block.type == "server_tool_use":
                log.info("%s -> %s %s", record.agent, block.name, _preview(getattr(block, "input", {}))
    ↪ )
        if response.stop_reason == "refusal":
            details = getattr(response, "stop_details", None)
            raise AgentRefusedError(f"{record.agent} refused: {getattr(details, 'category', None)}")
        messages.append({"role": "assistant", "content": response.content})
        record.server_tool_calls += sum(1 for b in response.content if b.type == "server_tool_use")
        if response.stop_reason == "pause_turn":  # a long server-tool turn: resend to resume it
            continue
        if response.stop_reason == "max_tokens":
            raise RuntimeError(f"{record.agent}: response hit max_tokens ({MAX_TOKENS})")
        if response.stop_reason != "tool_use":
            record.final_text = "\n".join(b.text for b in response.content if b.type == "text").strip()
            log.info("%s finished after %d turns, %d tool calls: %s", record.agent, record.turns,
                     len(record.tool_calls), _preview(record.final_text, 400))
            return
        calls = [b for b in response.content if b.type == "tool_use"]
        results = await asyncio.gather(*(self._call_tool(record, sessions, b) for b in calls))
        messages.append({"role": "user", "content": list(results)})  # all results in one message
    raise RuntimeError(f"{record.agent}: no final answer after {self.max_turns} turns")
```

Code 35: One streamed model turn

python/nabia/ai/agents/runtime.py:240-256

```
async def _create(self, live: LiveOutput, request: dict[str, Any]) -> Any:
    """One model turn, streamed when the client supports it, so the output is
    logged as it is written; clients with only ‘‘create‘‘ (tests) are awaited whole."""
    messages_api = self.client.beta.messages
    if not hasattr(messages_api, "stream"):
        return await messages_api.create(**request)
    async with messages_api.stream(**request) as stream:
        async for event in stream:
            if event.type == "content_block_delta":
                delta = event.delta
                if delta.type == "text_delta":
                    live.add("text", delta.text)
                elif delta.type == "thinking_delta":
                    live.add("thinking", delta.thinking)
        response = await stream.get_final_message()
    live.flush()
    return response
```

Code 36: Live logging of streamed output

python/nabia/ai/agents/runtime.py:74-111

```
class LiveOutput:
    """Logs a streamed reply while it is written: a line per paragraph, or every
    few seconds / few hundred characters for long lines, so a job log follows
    the model as it thinks and writes."""

    def __init__(self, agent: str, max_chars: int = 400, max_seconds: float = 3.0) -> None:
        self.agent = agent
        self.max_chars = max_chars
        self.max_seconds = max_seconds
        self.kind = ""
        self.buffer = ""
        self.last = time.monotonic()
        self.chars = 0

    def add(self, kind: str, text: str) -> None:
        if kind != self.kind:
            self.flush()
            self.kind = kind
        self.buffer += text
        self.chars += len(text)
        while "\n" in self.buffer:
            line, self.buffer = self.buffer.split("\n", 1)
```

```
            self._emit(line)
        slow = len(self.buffer) >= self.max_chars // 5 and time.monotonic() - self.last >= self.
    ↪ max_seconds
        if len(self.buffer) >= self.max_chars or slow:  # long lines in pieces; slow streams every few
    ↪ seconds
            self._emit(self.buffer)
            self.buffer = ""

    def flush(self) -> None:
        if self.buffer:
            self._emit(self.buffer)
        self.buffer = ""

    def _emit(self, text: str) -> None:
        self.last = time.monotonic()
        if text.strip():
            label = "thinking" if self.kind == "thinking" else "writes"
            model_log.info("%s %s: %s", self.agent, label, text.strip())
```

Code 37: Executing one tool call on its MCP server
`python/nabia/ai/agents/runtime.py:258-276`

```
async def _call_tool(self, record: AgentRun, sessions: dict[str, ClientSession], block: Any) -> dict[
    ↪ str, Any]:
    session = sessions.get(block.name)
    log.info("%s -> %s(%s)", record.agent, block.name, _preview(dict(block.input)))
    try:
        if session is None:
            raise ValueError(f"unknown tool {block.name}")
        result = await session.call_tool(block.name, dict(block.input))
        text = "\n".join(c.text for c in result.content if getattr(c, "type", "") == "text")
        if not text and getattr(result, "structured_content", None) is not None:
            text = json.dumps(result.structured_content)
        is_error = bool(result.is_error)
    except Exception as error:  # a broken server must not kill the agent: report it as a tool error
        text, is_error = f"{type(error).__name__}: {error}", True
    record.tool_calls.append(ToolCall(block.name, dict(block.input), is_error, text[:2000]))
    if is_error:
        log.warning("%s <- %s failed: %s", record.agent, block.name, _preview(text, 300))
    else:
        log.info("%s <- %s: %s", record.agent, block.name, _preview(text))
    return {"type": "tool_result", "tool_use_id": block.id, "content": text or "(no output)", "is_error
    ↪ ": is_error}
```

16.3 Providers

Claude (`provider: "anthropic"`) uses `claude-opus-5-5` through the Anthropic SDK with the options above.

Local model (`provider: "ollama"`) uses an adapter that exposes the same interface (`beta.messages.create` and `.stream`) and translates to Ollama's chat API:

- the system prompt becomes a system message, and tools become function tools;
- assistant `tool_use` blocks become `tool_calls`, and `tool_result` blocks become `tool` messages (errors prefixed `ERROR:`);
- streamed NDJSON chunks become text and thinking deltas, and the merged reply becomes the final message;
- Claude-only options are ignored, and hosted tools are dropped, so local news comes from RSS.

Model options (temperature 0.2, a 32k context) come from the configuration.

Code 38: Ollama streaming as Anthropic-style events
`python/nabia/ai/agents/ollama.py:50-80`

```python
class _Stream:
    """A streamed reply: an async context manager and iterator of delta events."""

    def __init__(self, owner: OllamaClient, kwargs: dict[str, Any]) -> None:
        self._owner = owner
        self._kwargs = kwargs
        self._final: SimpleNamespace | None = None

    async def __aenter__(self) -> _Stream:
        return self

    async def __aexit__(self, *exc: Any) -> None:
        return None

    async def __aiter__(self) -> AsyncIterator[SimpleNamespace]:
        data: dict[str, Any] = {}
        async for chunk in self._owner._chat(self._kwargs, stream=True):
            message = chunk.get("message") or {}
            for kind, key in (("thinking", "thinking"), ("text", "content")):
                if message.get(key):
                    yield SimpleNamespace(type="content_block_delta",
                                          delta=SimpleNamespace(type=f"{kind}_delta", **{kind: message[
    ↪ key]}))
            _merge(data, chunk)
        self._final = self._owner._response(data)

    async def get_final_message(self) -> SimpleNamespace:
        if self._final is None:
            async for _ in self:
                pass
        assert self._final is not None
        return self._final
```

Code 39: Ollama replies translated to content blocks
`python/nabia/ai/agents/ollama.py:183-209`

```python
def _response(self, data: dict[str, Any]) -> SimpleNamespace:
    message = data.get("message", {})
    content: list[SimpleNamespace] = []
    text = (message.get("content") or "").strip()
    if text:
        content.append(SimpleNamespace(type="text", text=text))
    for call in message.get("tool_calls") or []:
        function = call.get("function", {})
        arguments = function.get("arguments") or {}
        if isinstance(arguments, str):  # some models return a JSON string
            try:
                arguments = json.loads(arguments)
            except json.JSONDecodeError:
                arguments = {"_unparsed": arguments}
        call_id = call.get("id") or f"call_{next(self._ids)}"
        self._tool_names[call_id] = function.get("name", "")
        content.append(SimpleNamespace(type="tool_use", id=call_id, name=function.get("name", ""),
    ↪ input=arguments))
    if any(b.type == "tool_use" for b in content):
        stop_reason = "tool_use"
    elif data.get("done_reason") == "length":
        stop_reason = "max_tokens"
    else:
        stop_reason = "end_turn"
    usage = SimpleNamespace(input_tokens=data.get("prompt_eval_count", 0), output_tokens=data.get("
    ↪ eval_count", 0),
                            cache_read_input_tokens=0, cache_creation_input_tokens=0)
    return SimpleNamespace(content=content, stop_reason=stop_reason, usage=usage, stop_details=None,
                           model=data.get("model", self.model))
```

Code

request translation — `python/nabia/ai/agents/ollama.py:119-153`

HTTP call, streamed or not — `python/nabia/ai/agents/ollama.py:162-181`

17 Agent prompts

17.1 Structure and templating

Every system prompt has the same five parts:

1. the agent's role in a cross-asset trend portfolio;
2. the *as-of rule*;
3. the portfolio universe;
4. the job;
5. a numbered workflow that names the tools to use, ending with the expected output.

`AgentTeam._fill` fills the templates:

`{as_of}` the decision date. In live mode it is today.

`{universe}` one line per portfolio symbol, `- SYMBOL: name, asset_class`, from the asset table.

`{as_of_rule}` the rule below, with the date filled in.

`{news_days}` the news window of the execution agent (7 by default).

`{research}` the news analyzer's research step. With web access (Claude) it adds web search and fetch. With a local model it uses the RSS headlines only.

`{mode_note}` the backtest note for non-live runs, and empty otherwise.

Code 40: The as-of rule (in every system prompt)
`python/nabia/ai/agents/team.py:17-20`

```
Today is {as_of}. Treat that date as the present: base every judgement on what the tools return (they
    ↪ only see data up to {as_of}) and never on knowledge of events after {as_of}.
```

Code 41: Template filling
`python/nabia/ai/agents/team.py:135-139`

```
def _fill(self, template: str, as_of: dt.date, **extra: Any) -> str:
    values = {"as_of": as_of.isoformat(), "universe": self._universe(),
              "news_days": self.config.news_lookback_days, **extra}
    values["as_of_rule"] = AS_OF_RULE.format(**values)
    return template.format(**values)
```

Code

the universe lines — `python/nabia/ai/agents/team.py:127-133`

17.2 News analyzer

The news analyzer runs with effort `medium`, the `news` server (writable) and, on Claude, web search and web fetch (each capped at `news_max_searches` uses). Its user message is *"Collect and store the market-moving news for {as_of}."* The prompt defines the sentiment and relevance rubric (§20) and requires real URLs and dates.

Code 42: News analyzer system prompt (template)
`python/nabia/ai/agents/team.py:22-43`

```
You are the news analyst of a systematic cross-asset trend portfolio.

{as_of_rule}

Portfolio universe (symbol: name, asset class):
{universe}

Your job: find the financial news published on {as_of} and the previous day that matters for these
assets, and store it in the news database. Cover central banks and rates, inflation and growth data,
equity markets, energy and metals, and crypto; prefer primary and reputable sources.

How to work:
1. Call query_news for {as_of} (days=2) to see what is already stored; do not store it again.
2. {research}
   Aim for 8-20 distinct, material articles, not every headline: skip stories with no plausible
   effect on these assets.
3. Store them with store_news_articles (in batches is fine). Use only URLs and dates you actually saw
   in the feed items or search results; never invent them. topic: the main driver. asset_classes and
     ↪ symbols: what the story
   moves (symbols only from the universe above). sentiment: expected price impact on the affected
   assets, -1 clearly bearish to +1 clearly bullish (0 neutral or mixed). relevance: 1 for
   portfolio-wide shocks, 0.5 for one asset class, lower for minor items.
4. Finish with five lines summarizing the dominant themes and their direction.
```

17.3 Stats analyzer

The stats analyzer runs with effort `high` and the `stats` server (writable). Its user message is *"Analyze the portfolio assets as of {as_of} and save the stats report."* It must:

- judge each asset's trend, volatility regime and diversification;
- propose a tilt in $[-1, 1]$ proportionate to the evidence;
- save a schema-validated report (`StatsReport`) covering every symbol.

Code 43: Stats analyzer system prompt (template)
`python/nabia/ai/agents/team.py:45-66`

```
You are the quantitative analyst of a systematic cross-asset trend portfolio that allocates
risk across the assets below with risk budgets (each asset's share of portfolio risk).

{as_of_rule}

Portfolio universe:
{universe}

Your job: analyze the statistical state of these assets as of {as_of} and save a structured report
for the portfolio manager.

How to work:
1. Look at market_regime, trend_signals, rolling_volatility and correlation_matrix (60-day, and
   252-day for context). Use get_stats_report to see the previous report for continuity.
2. For each asset judge: trend (sign, strength and persistence across horizons), volatility regime
   (level and percentile vs its last year), and diversification (correlation with the others,
   clusters of assets that move together).
3. Suggest a budget tilt per asset in [-1, 1]: positive for persistent uptrends with contained
   volatility and diversification value; negative for downtrends, stressed volatility or redundant
   exposure. Tilts should be proportionate to the strength of the evidence; 0 when unclear.
4. Save the report with save_stats_report (every symbol in assets, numbers cited in each reason).
5. Finish with a short summary of the regime and the main tilts.
```

17.4 Execution agent

The execution agent runs with effort `high`, the `news` and `stats` servers read-only, and the `portfolio` server. Its user message is *"Decide the risk budgets for {as_of} and compute the*

target weights." The prompt fixes its decision policy:

- start from equal risk;
- tilt mainly on the statistical evidence, using news as a secondary input;
- respect the bounds;
- keep budgets close to the previous decision because trading costs money, and explain any large change.

Code 44: Execution agent system prompt (template)
`python/nabia/ai/agents/team.py:68-91`

```
You are the portfolio manager of a systematic cross-asset trend portfolio. The portfolio holds
the assets below with long-only risk budgeting: you set each asset's risk budget (its share of total
portfolio risk); the engine turns budgets into weights using the trailing covariance and scales the
portfolio to its volatility target.

{as_of_rule}

Portfolio universe:
{universe}

Your job: decide the risk budgets for {as_of} and compute the resulting target weights.

How to work:
1. Read get_portfolio_config (budget bounds, settings) and get_previous_decision.
2. Read the quantitative view with get_stats_report and its underlying signals if needed.
3. Read news_summary for {as_of} (last {news_days} days) and query_news for detail on material themes.
4. Decide budgets. Start from equal risk and tilt with evidence: primarily the statistical trend and
   risk picture; news as a secondary input that can confirm, temper or flag a risk the statistics do
   not show yet. Respect the bounds. Trading has costs: keep budgets close to the previous decision
   unless the evidence has changed materially, and explain any large change.
5. Call set_risk_budgets for {as_of} with a rationale that cites the evidence behind each tilt, then
   compute_target_weights for {as_of}.
6. Finish with a table of budgets and weights and a three-sentence rationale.
{mode_note}
```

Code 45: Backtest note, appended for non-live runs (`{mode_note}`)
`python/nabia/ai/agents/team.py:93-95`

```
This is a historical simulation: news is only what was stored before {as_of}, and there is no web
access. Decide exactly as you would have on {as_of}.
```

Code 46: How each agent is called: prompts, user messages, servers, effort
`python/nabia/ai/agents/team.py:141-177`

```
async def collect_news(self, as_of: dt.date | None = None) -> AgentRun:
    """News analyzer: RSS feeds (and web search on Claude) -> news database.
    Live use only: feeds and the web are not point-in-time."""
    as_of = as_of or dt.date.today()
    if self.config.provider == "anthropic":
        web = [
            {"type": "web_search_20260209", "name": "web_search", "max_uses": self.config.
    ↪ news_max_searches},
            {"type": "web_fetch_20260209", "name": "web_fetch", "max_uses": self.config.
    ↪ news_max_searches},
        ]
        research = ("Call fetch_news_feeds for the latest headlines from financial news and central-
    ↪ bank\n"
                    "   feeds, then use web_search (and web_fetch on promising URLs when a snippet is
    ↪ not\n"
                    "   enough) to fill gaps and confirm the most market-moving stories.")
    else:
        web = []  # a local model has no web search: the RSS feeds are the source
        research = ("Call fetch_news_feeds for the latest headlines from financial news and central-
    ↪ bank\n"
```

```
                  "   feeds, and pick the stories that matter, judging them from their title and
    ↪ description.")
    return await self._run(
        "news_analyzer", as_of, system=self._fill(NEWS_SYSTEM, as_of, research=research),
        prompt=f"Collect and store the market-moving news for {as_of}.",
        servers=[MCPServerSpec(NEWS_SERVER)], server_tools=web, effort=self.config.effort.get("news", "
    ↪ medium"))

async def analyze(self, as_of: dt.date) -> AgentRun:
    """Stats analyzer: statistics -> saved stats report."""
    return await self._run(
        "stats_analyzer", as_of, system=self._fill(STATS_SYSTEM, as_of),
        prompt=f"Analyze the portfolio assets as of {as_of} and save the stats report.",
        servers=[MCPServerSpec(STATS_SERVER)], effort=self.config.effort.get("stats", "high"))

async def decide(self, as_of: dt.date) -> AgentRun:
    """Execution agent: news + stats -> risk budgets -> target weights."""
    note = BACKTEST_NOTE.format(as_of=as_of.isoformat()) if self.run != "live" else ""
    return await self._run(
        "execution", as_of, system=self._fill(EXECUTION_SYSTEM, as_of, mode_note=note),
        prompt=f"Decide the risk budgets for {as_of} and compute the target weights.",
        servers=[MCPServerSpec(NEWS_SERVER, read_only=True), MCPServerSpec(STATS_SERVER, read_only=True
    ↪ ),
                 MCPServerSpec(PORTFOLIO_SERVER)],
        effort=self.config.effort.get("execution", "high"))
```

18 Statistics computed for the agents

All statistics use daily close prices up to `as_of`, over `stats_history_days` (400) calendar days. Volatilities are annualized with $\sqrt{252}$, standard deviations are sample standard deviations, and returns are simple returns.

Aligned returns keep the dates where all assets have a price, then take returns. They are used for correlations only. Single-asset statistics use each asset's own history.

Correlation is the Pearson correlation matrix ρ of the last w aligned returns (60 by default, at least 3). The *average correlation* is

$$\bar{\rho} = \frac{1}{n(n-1)} \sum_{i \neq j} \rho_{ij}.$$

Rolling volatility is $\sigma^{(w)} = \mathrm{sd}(r_{T-w+1..T})\sqrt{252}$ for $w \in \{20, 60, 120\}$. The *volatility percentile* is computed from the series v_s of rolling 20-day standard deviations over the last 252 dates:

$$\pi = \frac{1}{|v|} \sum_{s} \mathbf{1}[v_s \leq v_T].$$

So $\pi = 1$ means the highest of the year.

Past returns $P_T / P_{T-n} - 1$ for $n \in \{21, 63, 126, 252\}$ observations (1, 3, 6 and 12 months).

Moving-average distance $P_T / \overline{P}_n - 1$, where $\overline{P}_n$ is the mean of the last $n \in \{50, 200\}$ prices.

Volatility-scaled 12-1 momentum

$$m = \frac{P_{T-21}/P_{T-252} - 1}{\mathrm{sd}(r_{T-251..T})\sqrt{252}}.$$

It is the 11-month return ending one month ago, which skips the short-term reversal, measured in units of the asset's own annual volatility.

Drawdowns over the last 253 prices: $D_t = 1 - P_t / \max_{s \le t} P_s$. The tools report the current value and the maximum.

Market regime combines five numbers:

- $\bar{\rho}$ over 60 and over 252 aligned returns;
- the median volatility percentile;
- the share of assets above their 200-day average;
- the share of assets with a positive 12-month return.

Code 47: Trend signals

python/nabia/ai/analytics.py:51-75

```python
def trend_signals(prices: pd.DataFrame) -> pd.DataFrame:
    """Per symbol: past returns (1m, 3m, 6m, 12m), distance from the 50- and
    200-day moving averages, a volatility-scaled 12-1 momentum score, and the
    current and 1-year maximum drawdowns."""
    out = {}
    for symbol in prices.columns:
        p = prices[symbol].dropna()
        last = p.iloc[-1]
        row: dict[str, float] = {}
        for label, n in (("ret_1m", 21), ("ret_3m", 63), ("ret_6m", 126), ("ret_12m", 252)):
            row[label] = float(last / p.iloc[-n - 1] - 1) if len(p) > n else float("nan")
        for n in (50, 200):
            row[f"vs_ma{n}"] = float(last / p.tail(n).mean() - 1) if len(p) >= n else float("nan")
        r = p.pct_change().dropna()
        vol = r.tail(252).std() * np.sqrt(PERIODS_PER_YEAR)
        if len(p) > 252 and vol > 0:
            row["momentum_12_1"] = float((p.iloc[-22] / p.iloc[-253] - 1) / vol)  # skip the last month
        else:
            row["momentum_12_1"] = float("nan")
        year = p.tail(253)
        drawdown = 1 - year / year.cummax()
        row["drawdown"] = float(drawdown.iloc[-1])
        row["max_drawdown_1y"] = float(drawdown.max())
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 48: Volatilities and their percentile

python/nabia/ai/analytics.py:36-48

```python
def rolling_volatility(prices: pd.DataFrame, windows: tuple[int, ...] = (20, 60, 120)) -> pd.DataFrame:
    """Per symbol: annualized volatility over each window (its own returns, not
    aligned with the other assets), plus where the 20-day volatility sits in
    its last year of history (percentile, 0..1)."""
    out = {}
    for symbol in prices.columns:
        r = prices[symbol].dropna().pct_change().dropna()
        row = {f"vol_{w}d": float(r.tail(w).std() * np.sqrt(PERIODS_PER_YEAR)) if len(r) >= w else
    ↪ float("nan")
               for w in windows}
        rolling = r.rolling(20).std().dropna().tail(PERIODS_PER_YEAR)
        row["vol_20d_percentile_1y"] = float((rolling <= rolling.iloc[-1]).mean()) if len(rolling) else
    ↪  float("nan")
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 49: Market regime

python/nabia/ai/analytics.py:78-91

```python
def market_regime(prices: pd.DataFrame) -> dict[str, float]:
    """Cross-asset summary: average correlation now (60d) vs over the last year,
    median volatility percentile, and the share of assets in an uptrend."""
    corr_now = average_correlation(correlation_matrix(prices, 60))
    corr_year = average_correlation(correlation_matrix(prices, 252)) if len(aligned_returns(prices)) >=
    ↪  60 else float("nan")
```

```
    vols = rolling_volatility(prices)
    trends = trend_signals(prices)
    return {
        "avg_correlation_60d": corr_now,
        "avg_correlation_1y": corr_year,
        "median_vol_percentile_1y": float(vols["vol_20d_percentile_1y"].median()),
        "share_above_ma200": float((trends["vs_ma200"] > 0).mean()),
        "share_positive_12m": float((trends["ret_12m"] > 0).mean()),
    }
```

Code

correlation matrix — `python/nabia/ai/analytics.py:21-26`
average correlation — `python/nabia/ai/analytics.py:29-33`

19 From decisions to positions

19.1 Budget normalization within bounds

The execution agent proposes positive *relative* budgets $\tilde{b}$. The tool `set_risk_budgets` maps them to budgets with $\sum_i b_i = 1$ and $b_{\min} \le b_i \le b_{\max}$:

1. Let F be the set of fixed assets, initially empty. Scale the free assets to fill the rest: $b_i = \tilde{b}_i \,(1 - \sum_{j \in F} b_j) / \sum_{j \notin F} \tilde{b}_j$.
2. Fix every free asset below $b_{\min}$ at $b_{\min}$, and every one above $b_{\max}$ at $b_{\max}$.
3. Repeat until nothing is out of bounds. If every asset ends fixed, renormalize the fixed values.

This is well defined whenever $n b_{\min} \le 1 \le n b_{\max}$, which validation guarantees. Each pass fixes at least one asset, so it ends after at most n passes. The free assets keep their proposed *ratios*, so the rule changes the agent's intent as little as the bounds allow.

The tool returns the budgets and how far the bounds moved each one. `preview_budgets` shows the same result without saving it.

Code 50: Budget normalization
`python/nabia/ai/analytics.py:107-139`

```
def project_budgets(budgets: dict[str, float], min_budget: float, max_budget: float) -> dict[str, float
    ↪ ]:
    """Normalizes positive budgets to sum to 1 with every value in [min_budget, max_budget].

    Values beyond a bound are fixed at that bound and the rest rescaled to fill
    the remainder, repeating until nothing is out of bounds.
    """
    if not budgets:
        raise ValueError("no budgets")
    if any(not np.isfinite(v) or v <= 0 for v in budgets.values()):
        raise ValueError("budgets must be positive numbers")
    n = len(budgets)
    if not (min_budget * n <= 1.0 + 1e-12 and max_budget * n >= 1.0 - 1e-12):
        raise ValueError("bounds are infeasible for this number of assets")
    fixed: dict[str, float] = {}
    free = dict(budgets)
    while True:
        remaining = 1.0 - sum(fixed.values())
        total = sum(free.values())
        scaled = {k: v * remaining / total for k, v in free.items()}
        low = {k for k, v in scaled.items() if v < min_budget - 1e-12}
        high = {k for k, v in scaled.items() if v > max_budget + 1e-12}
        if not low and not high:
            result = {**fixed, **scaled}
            return {k: result[k] for k in budgets}
        for k in low:
            fixed[k] = min_budget
```

```
            free.pop(k)
        for k in high:
            fixed[k] = max_budget
            free.pop(k)
        if not free:
            result = {k: fixed[k] / sum(fixed.values()) for k in budgets}
            return result
```

Code 51: The decision tool
python/nabia/ai/servers/portfolio.py:45-58

```
@tool(server)
def set_risk_budgets(as_of: str, budgets: dict[str, float], rationale: str) -> dict:
    """Decide the risk budgets for as_of: one positive number per portfolio symbol
    (relative sizes; normalized to sum to 1 and bounded to [min_budget, max_budget]).
    The rationale should cite the evidence (stats, news) behind each tilt."""
    end = parse_as_of(as_of)
    _check_symbols(ctx, budgets)
    if len(rationale.strip()) < 20:
        raise ValueError("give a rationale that explains the budgets")
    projected = analytics.project_budgets(budgets, ctx.config.min_budget, ctx.config.max_budget)
    ctx.store.save_decision(ctx.run, end, projected, rationale.strip(), decided_by=f"agent:{ctx.config.
    ↪ model_label}")
    changed = {s: round(projected[s] - budgets[s] / sum(budgets.values()), 4) for s in projected}
    return clean({"as_of": end.isoformat(), "budgets": projected,
                  "adjusted_by_bounds": {s: d for s, d in changed.items() if abs(d) > 1e-6}}, 4)
```

19.2 Target weights

`compute_target_weights` loads at least $\max(3W + 30, 120)$ calendar days of prices up to `as_of`. It runs `nabia.latest_weights` with the decided budgets, which is the engine of §9 restricted to the last rebalance. It stores the weights and leverage with the decision.

Each decision records who made it, as `trend_rule` or as `agent:<model>` (for example `agent:ollama:qwen3.8:latest`). Runs on different models can therefore be compared.

Code 52: Budgets to target weights through the C++ engine
python/nabia/ai/servers/portfolio.py:82-93

```
def compute_weights(ctx: ServerContext, as_of: dt.date, budgets: dict[str, float]) -> dict:
    """Runs nabia.latest_weights on prices up to as_of and stores the weights with the decision."""
    import nabia  # the C++ extension, imported on use

    days = int(ctx.config.portfolio.get("covariance_window", 60)) * 3 + 30
    prices = ctx.prices(as_of, days=max(days, 120))
    target = nabia.latest_weights(prices, {**ctx.config.portfolio, "budgets": budgets})
    weights = {s: float(w) for s, w in target.weights.items()}
    ctx.store.save_weights(ctx.run, as_of, weights, target.leverage)
    return clean({"as_of": as_of.isoformat(), "priced_on": target.date.date().isoformat(),
                  "weights": weights, "risk_contributions": target.risk_contributions.to_dict(),
                  "leverage": target.leverage, "converged": target.converged}, 4)
```

19.3 The trend rule baseline

A transparent, model-free policy is the benchmark that agent policies must beat:

$$\tilde{b}_i = \max\bigl(1 + s \cdot \mathrm{clip}(m_i, -1, 1),\ 10^{-6}\bigr), \qquad s = 0.5,$$

followed by the same bounded normalization. A missing momentum counts as 0. A strong uptrend ($m_i \geq 1$) gets 1.5× the equal budget before normalization, and a strong downtrend 0.5×. Over 2019–2026, on the default universe with monthly decisions, it reached a Sharpe ratio of 0.96 against 0.92 for equal risk.

Code 53: The trend rule
python/nabia/ai/analytics.py:94-104

```
def trend_tilted_budgets(prices: pd.DataFrame, strength: float = 0.5, min_budget: float = 0.02,
                         max_budget: float = 0.40) -> dict[str, float]:
    """A transparent rule-based policy: equal risk budgets tilted towards assets
    with positive volatility-scaled momentum, bounded and normalized.

    budget_i ~ 1 + strength * clip(momentum_12_1_i, -1, 1). Used as a baseline
    for the agents and as a deterministic decision rule in tests.
    """
    momentum = trend_signals(prices)["momentum_12_1"].fillna(0.0).clip(-1.0, 1.0)
    raw = (1.0 + strength * momentum).clip(lower=1e-6)
    return project_budgets(raw.to_dict(), min_budget, max_budget)
```

Code

trend rule as a backtest policy — python/nabia/ai/pipeline.py:64-80

19.4 Backtest mode

Decision dates. The trading dates in [start, end]: every one for `D`, or the last of each week (`W-FRI`), month (`ME`) or quarter (`QE`).

Policies.

- `trend_rule`;
- `ollama` (agents on the local model);
- `claude` (agents on Claude);
- `agents` (agents on the configured provider).

Procedure.

1. At each date d, the policy decides budgets as of d. A date that already has a decision is skipped, so interrupted backtests resume.
2. The decisions form a budget schedule. Budgets decided at the close of d apply from the rebalance at d on.
3. One continuous backtest runs with the schedule, together with an equal-risk baseline over the same period. The run's policy, period and metrics are stored.

The decision frequency is not the trading frequency: trades follow the parameter `rebalance_frequency`, and each rebalance uses the latest decision.

Code 54: Decision dates
python/nabia/ai/pipeline.py:53-61

```
def decision_dates(index: pd.DatetimeIndex, start: dt.date, end: dt.date, frequency: str = "ME") ->
    ↪ list[dt.date]:
    """Decision dates within [start, end]: every trading date for "D" (daily),
    otherwise the last trading date of each period (pandas offset alias: "ME"
    month end, "W-FRI" weekly, "QE" quarter end)."""
    dates = pd.Series(index, index=index)
    dates = dates[(dates >= pd.Timestamp(start)) & (dates <= pd.Timestamp(end))]
    if frequency.upper() in ("D", "B"):
        return [d.date() for d in dates]
    return [d.date() for d in dates.groupby(pd.Grouper(freq=frequency)).max().dropna()]
```

Code 55: Decisions to a backtest, against equal risk
`python/nabia/ai/pipeline.py:180-196`

```python
def evaluate_decisions(config: AIConfig, run: str, start: dt.date, end: dt.date,
                       prices: pd.DataFrame | None = None) -> AgentBacktest:
    """Backtest of the decisions already stored for ‘run‘ between start and end
    (no new decisions), against equal risk budgets."""
    import nabia

    prices = _backtest_prices(config, start, end) if prices is None else prices
    decisions = [d for d in AgentStore(config.db_url).decisions(run)
                 if start.isoformat() <= d["as_of"] <= end.isoformat()]
    if not decisions:
        raise ValueError(f"run {run!r} has no decisions between {start} and {end}")
    schedule = pd.DataFrame([d["budgets"] for d in decisions],
                            index=pd.DatetimeIndex([d["as_of"] for d in decisions], name="date"))[
     ↪ config.symbols]
    params = dict(config.portfolio)
    result = nabia.backtest(prices, params, budget_schedule=schedule)
    baseline = nabia.backtest(prices, params)
    return AgentBacktest(result, baseline, schedule, decisions)
```

Code

the backtest loop — `python/nabia/ai/pipeline.py:132-171`
policy to decider — `python/nabia/ai/pipeline.py:101-105`
agent policy (stats, then execution) — `python/nabia/ai/pipeline.py:108-115`

Look-ahead control. The tools cannot see the future. In backtests, news comes only from the database (no web access), and the prompt asks the model to treat the decision date as today. One leak cannot be closed by tooling: the language model was trained on data up to its knowledge cutoff and may recognize market history before it. Agent backtests inside that window are therefore optimistic. They are fully out of sample only after the cutoff.

20 News: collection, sentiment and relevance

20.1 Collection

The news analyzer runs once a day in live mode.

1. It calls `query_news` for the last two days to see what is already stored.
2. It gathers candidate stories.
   - `fetch_news_feeds` reads ten public feeds: MarketWatch, CNBC (two), Financial Times, Investing.com, CoinDesk, OilPrice, the Federal Reserve, the ECB and the Bank of England. The list is configurable. The tool parses RSS and Atom, keeps items newer than `max_age_days`, removes URLs already stored, and returns the newest first, with HTML stripped from descriptions. A broken feed is reported without hiding the others.
   - On Claude, web search and fetch also fill gaps and confirm stories.
3. It keeps 8–20 material stories, classifies them, and stores them with `store_news_articles`, keyed by URL (an upsert).

Code 56: RSS and Atom headlines without stored articles
`python/nabia/ai/servers/news.py:71-90`

```python
@tool(server)
def fetch_news_feeds(max_age_days: int = 2, max_items: int = 60) -> dict:
    """Latest headlines from the configured financial news RSS feeds (markets,
    economy, central banks, energy, crypto), newest first, skipping articles
```

```
    already stored. Use it to find candidate stories to classify and store."""
    cutoff = dt.datetime.now(dt.UTC) - dt.timedelta(days=max_age_days)
    items, errors = [], []
    for source, url in ctx.config.news_feeds.items():
        try:
            items += [i for i in _read_feed(source, url) if i["published"] is None or i["published"] >=
    ↪  cutoff]
        except Exception as error:  # one broken feed must not hide the others
            errors.append(f"{source}: {type(error).__name__}")
    known = ctx.store.known_urls([i["url"] for i in items])
    fresh, seen = [], set()
    for i in sorted(items, key=lambda i: i["published"] or cutoff, reverse=True):
        if i["url"] in known or i["url"] in seen:
            continue
        seen.add(i["url"])
        fresh.append({**i, "published": i["published"].date().isoformat() if i["published"] else None})
    return {"items": fresh[:max_items], "already_stored": len(known), "feed_errors": errors}
```

Code

feed parsing — `python/nabia/ai/servers/news.py:163-180`

20.2 How sentiment and relevance are produced

Sentiment and relevance are judgements made by the language model for each article. They are not computed by a statistical model or a word list. The news analyzer reads the headline and description, and on Claude possibly the article, then fills the `NewsArticle` schema following the rubric in its prompt:

Sentiment $s \in [-1, 1]$ is the *expected price impact on the affected assets*: -1 clearly bearish, $+1$ clearly bullish, 0 neutral or mixed. It is relative to the assets the story moves, not the tone of the text. A hawkish central bank is bearish for bonds even when the article reads upbeat.

Relevance $r \in [0, 1]$ is the importance of the story for a cross-asset portfolio: 1 for portfolio-wide shocks, about 0.5 for a story that moves one asset class, and lower for minor items.

Classification covers the main topic, the affected asset classes, and the portfolio symbols directly affected.

The server validates what the model produces before storing it:

- the schema bounds $s \in [-1, 1]$ and $r \in [0, 1]$;
- topic and asset classes from fixed lists;
- no future dates;
- symbols from the universe.

Invalid articles are rejected with a reason the model can act on.

Code 57: Article schema
`python/nabia/ai/servers/news.py:27-39`

```
class NewsArticle(BaseModel):
    """One news item, as extracted by the news analyzer."""

    url: str = Field(description="Canonical URL of the article (the unique key)")
    published_date: dt.date = Field(description="Publication date, YYYY-MM-DD")
    title: str
    source: str = Field(description="Publisher, e.g. Reuters, Bloomberg, FT")
    topic: Topic = Field(description="Main topic")
    asset_classes: list[AssetClass] = Field(description="Asset classes the news affects")
    symbols: list[str] = Field(default_factory=list, description="Portfolio symbols directly affected")
```

```
    sentiment: float = Field(ge=-1.0, le=1.0, description="Price impact on the affected assets: -1
    ↪ bearish .. +1 bullish")
    relevance: float = Field(ge=0.0, le=1.0, description="Importance for a cross-asset portfolio: 0 ..
    ↪ 1")
    summary: str = Field(description="Two or three factual sentences: what happened and why it matters"
    ↪ )
```

Code 58: Validation before storage
python/nabia/ai/servers/news.py:50-68

```
@tool(server)
def store_news_articles(articles: list[NewsArticle]) -> dict:
    """Store news articles (upsert by URL, so storing an article twice updates it).

    Only store articles with a real URL and publication date you have seen.
    Symbols must be portfolio symbols (see the news_summary universe)."""
    today = dt.date.today()
    universe = set(ctx.config.symbols)
    rejected = []
    accepted = []
    for a in articles:
        if a.published_date > today:
            rejected.append({"url": a.url, "reason": "published_date is in the future"})
        elif not set(a.symbols) <= universe:
            rejected.append({"url": a.url, "reason": f"unknown symbols {sorted(set(a.symbols) -
    ↪ universe)}"})
        else:
            accepted.append(a.model_dump())
    stored = ctx.store.store_news(accepted)
    return {"stored": stored, "rejected": rejected}
```

20.3 Aggregation

`news_summary` takes the articles A published in the d days up to and including `as_of` and groups them by topic, asset class and symbol. An article that affects several classes or symbols counts in each of their groups. For each group g it reports:

- the count;
- the maximum relevance;
- the **relevance-weighted sentiment**

$$S_g = \frac{\sum_{i \in g} s_i \, r_i}{\sum_{i \in g} r_i}.$$

When all relevances in a group are zero, the denominator is replaced by 1, so $S_g = 0$. S_g is a convex combination of the sentiments, so $S_g \in [-1, 1]$. Weighting by relevance lets one portfolio-wide story outweigh several minor ones. The summary also lists the ten most relevant headlines.

The web application's *News* page shows, over the selected filter, article counts per day and the simple (unweighted) average sentiment per topic.

Code 59: Relevance-weighted aggregation
python/nabia/ai/servers/news.py:101-134

```
@tool(server)
def news_summary(as_of: str, days: int = 7) -> dict:
    """Aggregate view of the news in the ‘days‘ days up to as_of: article counts and
    relevance-weighted sentiment per topic, asset class and symbol, plus the top headlines."""
    end = parse_as_of(as_of)
    articles = ctx.store.news(end - dt.timedelta(days=days - 1), end, limit=10_000)

    def aggregate(key_of) -> dict:
```

```
        groups: dict[str, list[dict]] = {}
        for a in articles:
            for key in key_of(a):
                groups.setdefault(key, []).append(a)
        out = {}
        for key, items in sorted(groups.items()):
            weight = sum(i["relevance"] for i in items) or 1.0
            out[key] = {
                "articles": len(items),
                "weighted_sentiment": sum(i["sentiment"] * i["relevance"] for i in items) / weight,
                "max_relevance": max(i["relevance"] for i in items),
            }
        return out

    top = sorted(articles, key=lambda a: a["relevance"], reverse=True)[:10]
    return clean({
        "as_of": end.isoformat(),
        "days": days,
        "universe": ctx.config.symbols,
        "articles": len(articles),
        "by_topic": aggregate(lambda a: [a["topic"]]),
        "by_asset_class": aggregate(lambda a: a["asset_classes"]),
        "by_symbol": aggregate(lambda a: a["symbols"]),
        "top_headlines": [{k: a[k] for k in ("published_date", "title", "source", "topic", "symbols",
                                             "sentiment", "relevance", "summary")} for a in top],
    }, 3)
```

Code

article queries — `python/nabia/ai/servers/news.py:92-99`
storage (upsert by URL) — `python/nabia/ai/db.py:149-162`
web News aggregates — `python/nabia/web/app.py:102-121`

20.4 Use and limitations

News is a *secondary* input by design. The execution agent tilts mainly on statistics and uses news to confirm, temper or flag risks the statistics do not show yet. Because the scores are model judgements:

- **They are not reproducible across models**, and only approximately across runs. Decisions record the model that made them.
- **They are uncalibrated.** A sentiment of 0.6 has no defined relation to a return. Using them as quantitative signals would require calibration first, for example a regression of next-day returns on S_g.
- **Coverage depends on the sources.** RSS headlines and descriptions are short, and web access gives fuller context.

Possible improvements:

- an ensemble of scoring models;
- a lexicon or finance-tuned classifier as a reproducible baseline;
- a calibration study of S_g against returns.

Part V
Operation

21 Web application and job runner

`nabia.web` is a FastAPI application with a single-page JavaScript front end (Chart.js). Start it with `PYTHONPATH=python python -m nabia.web --config ai_config.json`; it binds to 127.0.0.1 by default.

Endpoint	Purpose
`GET /api/overview`	latest live decision, data coverage, recent jobs, model
`GET /api/news`	articles with filters, counts per day, sentiment by topic
`GET /api/runs`, `/api/runs/{run}/decisions`	runs and their decisions
`GET /api/runs/{run}/stats_report`	a run's stats report as of a date
`GET /api/agent_runs`, `/api/agent_runs/{id}`	agent transcripts (tool calls, usage)
`GET /api/backtests/{run}`	equity, drawdown, metrics, schedule, weights vs equal risk
`GET /api/jobs`, `/api/jobs/{id}`	jobs and their status
`GET /api/jobs/{id}/logs?after=`	job log lines after a given id (tailing)
`POST /api/jobs`	start a job: `download`, `news`, `daily`, `backtest`

Jobs. Jobs run one at a time, in order, in the server's event loop. SQLite allows one writer, and agent runs are costly. Their status is stored in the database. Blocking work runs in worker threads so the server stays responsive:

- downloads;
- the trend rule;
- the final C++ backtest.

Live logs. A logging handler stores every record emitted while a job runs, under that job's id. The id lives in a *context variable*, which follows the job into its asyncio tasks and worker threads, so concurrent web requests never leak into a job's log. The log therefore contains:

- the pipeline's steps;
- each agent turn and each tool call, with input and output previews;
- the model's text and thinking as it streams. Long lines arrive in pieces of about 400 characters, and slow streams every few seconds.

Chatty libraries contribute only warnings and errors. While the job runs, the front end asks every second for the lines after the last one it has, and shows the model's streamed text and thinking in their own styles.

Code 60: Per-job log capture
`python/nabia/web/jobs.py:36-53`

```
class JobLogHandler(logging.Handler):
    """Stores the log records emitted while a job runs, under that job's id."""

    def __init__(self, store: AgentStore) -> None:
        super().__init__(logging.INFO)
        self.store = store
        self.setFormatter(logging.Formatter("%(message)s"))

    def emit(self, record: logging.LogRecord) -> None:
```

```
        job_id = current_job.get()
        if job_id is None:
            return
        if record.name.startswith(_QUIET) and record.levelno < logging.WARNING:
            return
        try:
            self.store.add_job_log(job_id, record.levelname, record.name, self.format(record))
        except Exception:  # logging must never break the job
            self.handleError(record)
```

Code 61: Running a job with its log context
`python/nabia/web/jobs.py:105-119`

```
async def _run(self, job_id: int, kind: str, params: dict[str, Any]) -> None:
    async with self._lock:
        token = current_job.set(job_id)
        try:
            self.store.update_job(job_id, status="running", message="started")
            log.info("job #%s: %s %s", job_id, kind, params or "")
            message = await getattr(self, f"_{kind}")(job_id, params)
            log.info("job #%s done: %s", job_id, message)
            self.store.update_job(job_id, status="done", message=message)
        except Exception as error:
            log.exception("job #%s failed", job_id)  # with the traceback, in the job log
            self.store.update_job(job_id, status="failed", error="".join(
                traceback.format_exception_only(error)).strip())
        finally:
            current_job.reset(token)
```

Code

application and endpoints — `python/nabia/web/app.py:54-217`
job runner — `python/nabia/web/jobs.py:61-152`
backtest job — `python/nabia/web/jobs.py:142-152`
front-end log viewer — `python/nabia/web/static/app.js:204`

22 Verification

C++ (Catch2): 88 test cases.

- Results against closed forms: inverse volatility for uncorrelated equal budgets, Gaussian VaR and ES, SES and Kalman steady states, filter gains at the band edges.
- Optimizer results against known optima, with constraints.
- Accounting identities to 10^{-15}.
- No look-ahead, thread-count independence, and invalid input.

The suites run in debug, under AddressSanitizer with UndefinedBehaviorSanitizer, and the multithreaded code under ThreadSanitizer.

Python (pytest): offline by default.

- The analytics, and the three MCP servers in process (including the no-look-ahead check).
- The agent loop over real stdio MCP servers with a scripted model (parallel calls, tool errors, paused turns, refusals).
- The Ollama translation and streaming against a mock transport, and live log chunking.
- RSS parsing, the pipeline, and the web API with job logs.

Opt-in tests download live market data and run the execution agent end to end on a local model.

This document Its excerpts, prompts, tool catalogue and configuration table are extracted at build time. A listed function that no longer exists stops the build.

23 Limitations and future work

- **Gaussian risk.** VaR and ES are Gaussian; fat tails are not modelled. A historical or Cornish–Fisher measure would fit the same interface.
- **Continuous futures** jump at rolls. Back-adjusted series would remove spurious returns.
- **Asynchronous closes** (Asia against US) bias same-day correlations downwards. Multi-day returns or synchronized prices would reduce the bias.
- **News scores** are uncalibrated model judgements (§20).
- **Agent backtests** before the model's knowledge cutoff are optimistic.
- **Jobs** run sequentially. A queue with a PostgreSQL backend would allow concurrent jobs.